



PROJECT REPORT

COURSE: LAB211 (Object-Oriented Programming
Lab)

PROJECT REPORT

Football Club - Player Management System Project

Instructor:	Nguyen Tri Thong
Student Name:	Tran Mai Anh Khoa
Student ID:	SE204826
Class:	SE2018

Contents

I	SYSTEM ANALYSIS & DESIGN DOCUMENTATION	4
1	Introduction	4
2	Problem Analysis	4
2.1	Nouns/Entities	4
2.2	Main Model Entity	5
2.3	Main Business Entity	5
2.4	Relationship of core entities	6
2.4.1	Player and Club	6
2.4.2	Model and Business	6
3	Computational Thinking	7
3.1	Purpose of Implementing Computational Thinking	7
3.2	Decomposition	7
3.2.1	Data decomposition	7
3.2.2	Functional Decomposition	8
3.2.3	Class decomposition	8
3.3	Pattern Recognition	9
3.3.1	Pattern rule	9
3.3.2	Business layer similarities	10
3.3.3	Design pattern implementation	10
3.4	Abstraction	11
3.5	Algorithm Design	14
4	OOP Design (UML)	25
4.1	Conceptual diagram	25
4.2	Class diagram decomposition	27
4.2.1	Player and club relationship	27
4.2.2	Model and Business relationship	28
4.2.3	Enum & Static factory pattern	29
4.2.4	Call back / Limit method access	30
4.2.5	Controller & OptionProcessor	31
4.2.6	Business class & ComparatorContainer	32
4.3	Full class diagram	33
4.3.1	Model & Inheritance	34
4.3.2	Business layer	35
4.3.3	Controller & Utils	36
5	Conclusion	37
II	PROJECT DEMO	37
6	Outer function demo	38

7 Function 0 – Exit the Program	38
8 Function 1 – List of all clubs	39
8.1 Happy case	39
8.2 Unhappy case	40
9 Function 2 – Add a new club	40
9.1 Happy case	40
9.2 Unhappy case	41
10 Function 3 – Search for a club by ID	43
10.1 Happy case	43
10.2 Unhappy case	44
11 Function 4 – Update a club by ID	45
11.1 Happy case	45
11.2 Unhappy case	46
12 Function 5 – List of all clubs with budget $\leq$ input value	50
12.1 Happy case	50
12.2 Unhappy case	51
13 Function 6 – List all players in ascending order of club names, if same sort by shirt number ascending	52
14 Function 7 – Search players by partial player name	53
14.1 Happy case	53
14.2 Unhappy case	53
15 Function 8 – Add a new player	54
15.1 Happy case	54
15.2 Unhappy case	55
16 Function 9 – Remove a player with ID	57
16.1 Happy case	57
16.2 Unhappy case	58
17 Function 10 – Update a player with an ID	58
17.1 Happy case	58
17.2 Unhappy case	60
18 Function 11 – List all players by a specific position	64
18.1 Happy case	64
18.2 Unhappy case	65
19 Function 12 – Save data to files	66
19.1 Happy case	66
19.2 Unhappy case	67

20 Function 13 – Load data from files	67
20.1 Happy case	67
20.2 Unhappy case	68

Part I

SYSTEM ANALYSIS & DESIGN DOCUMENTATION

1. Introduction

- **Introduction:**

In professional sports management, maintaining the integrity and consistency of data plays a decisive role in the operation of the entire organization. The project "**Football Club Management System**" focuses on designing an object-oriented data structure to manage information about Clubs and their list of Players.

- **Project Goal:**

This project purpose is to handle complex real-world business constraints of a Management System, Implement Computational Thinking (Decomposition, Pattern Recognition, Abstraction, Algorithm) and AI Reflection to reach the requirement of this project

2. Problem Analysis

2.1. Nouns/Entities

Based on the requirements of Lab 2, the system includes object classes, management classes, and supporting components as follows:

- **Main entities:** Club, Player (with subclasses: Goalkeeper, Defender, Midfielder, Forward, Winger).
- **Management classes (Managers):** Manager (Abstract), ClubManager, Player-Manager.
- **Control & Interface classes (Controller & UI):** Controller, ViewHandler, MenuContainer.
- **Data processing utilities (Utilities):** FileIOHandler, Inputter.
- **Supporting structural components:**
 - PlayerType (**Enum**),
 - ManagerType (**Enum**),
 - OptionProcessor(**Enum**),
 - HeaderType (**Enum**), Acceptable (**Interface**),
 - ClubAndPlayerConnection (**Interface**),
 - FunctionalInterfaceForCheckDuplicate(**Interface**)

2.2. Main Model Entity

1. Player:

- **Constraint:**

- *Player ID (Format: PXXXX)*: Must be unique.
- *Player Name*: Cannot be empty
- *Player Club ID*: Must contain the club in the list of data.
- *Player position*: One of these values:
Goalkeeper, Defender, Midfielder, Forward, Winger
- *Player Shirt Number*: Unique in a club and between 1 and 99

- **Evaluation:**

- The smallest unit of the main-entity category
- **Player ID & Player Club id**: Needs regex pattern to limit its format.
- **Player name**: Needs regex patterns to limit characters can be inputted.
- **Shirt number**: Needs a function to input and limit numbers in the range [1,99] along with a connection to the Player Club when changing the shirt number or deleting a player from the data list.
- **Player position**:
Goalkeeper, Defender, Midfielder, Forward, Winger can be child class of *Player*. Can use enum class to select which position should be created.

2. Club:

- **Constraint:**

- *Club ID(Format: CL-XXXX)*: Must be unique.
- *Club name* Cannot be empty.
- *Sponsor brand* Cannot be empty.
- *Budget* Must be a positive real number.

- **Evaluation:**

- A club can contain multiple players.
- **Club ID**: Needs regex pattern to limit its format.
- **Club Name & Sponsor Brand**: Needs regex patterns to limit characters can be inputted.
- **Club player shirt number data**: Club class need a list of data containing the data of which shirt number is own by which player (using player id as a connection) for easier to check existing shirt number or count the numbers of players in a single club.

2.3. Main Business Entity

1. *Manager <<Abstract>>*

- Handling modify main data and similar features of PlayerManager and ClubManager (Add, Update, Delete, Search, Contain ID Check). All of Read and the unique features will be abstract method and let its instance to override to use.
- Using data structure Hash Map with Key is ID and Value is Model Entities(Player, Club) for convenience of searching, optimize the searching performance with $O(1)$ time complexity.
- Implementing generic for value of Hash map for code reuseability.

2. *PlayerManager*

- A class inherited from Manager class.
- Handling read and unique feature such as:
 - Search by partial name.
 - Sort ascending by club name and shirt number field
 - Filter data by specific position.
 - Show data.
 - Save, load data.

3. *ClubManager*

- A class inherited from Manager class.
- Handling read and unique feature such as:
 - Filter data by budget of club (Input budget and show data of which club have its budget smaller or equal the inputted budget).
 - Show data.
 - Save, load data.

2.4. Relationship of core entities

2.4.1 Player and Club

- Based on the business rule, Player can only join one club at a time, Club can contain multiple player - Each player has its unique shirt number.
=> Relationship of Player and Club Entities is **1 - n**.

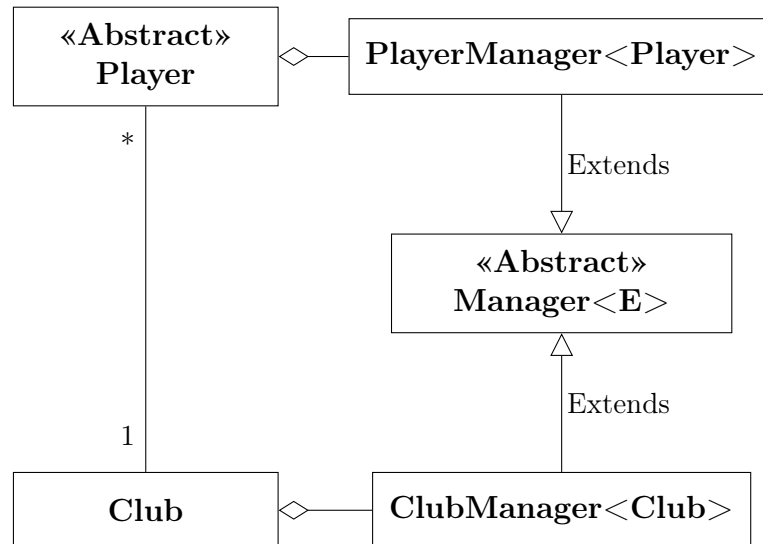


Conceptual diagram of Player and Club Relationship

2.4.2 Model and Business

- To store data of main model(Player, Club), Manager <<**Abstract**>> will contain a Hash Map to store each model information along with handle modifying data and the information will be displayed, sort, filter,..etc by PlayerManager/ClubManager.

- PlayerManager will store all the data of Player Entities which containing player informations.
- ClubManager will store all the data of Club Entities which containing club informations.



Conceptual diagram of Model and Business Entities Relationship.

3. Computational Thinking

3.1. Purpose of Implementing Computational Thinking

- In the development of the Football Club Management System, Computational Thinking (CT) serves as the core problem-solving methodology used to translate complex, real-world sports operations into a structured software architecture. Rather than treating programming merely as writing syntax.
- By applying this analytical framework, the complex mechanics of football administration are successfully transformed into logical, computational processes. This ensures that the final system is not only functionally accurate according to object-oriented programming principles but also highly scalable, secure against data corruption, and optimized for long-term maintenance.

3.2. Decomposition

3.2.1 Data decomposition

File	Entity	Role
clubs.txt	Club	Stores information about the clubs.
players.txt	Player	Describes information about the players belonging to the clubs.
log.txt	None	Write error log to file

- **Data Management**

- Save/Load data from file "clubs.txt" and "players.txt".
- Save changes before leave the program.
- Write log whenever an exception occur.

3.2.2 Functional Decomposition

Group	Functionality
File I/O	Read / Write data
Club	Add, Search, Update, Display, Filter, Check exist
Player	Add, Search, Update, Delete, Display, Filter, Check exist
Validation	Containing needed regex pattern, Club/Player ID exist, Shirt exist, Partial equal name, Decimal value, Decimal value in range
View handler	Menu, Table display, format name
Menu Container	Containing all option of multiple menu
Comparator container	Containing all of needed comparator for sorting
Connection Entities	Connection for synchronizing data between main entities
Input	Receive data from input and use Validation to ensure the accuracy of data
Selection	Select the behavior of data flow (Process option), select which entities should be created,..etc..
Controller	Connect and control all the entities, behavior to become a management system

3.2.3 Class decomposition

- All needed class in the system:
 1. **Controller group**
 - Main
 - Controller
 2. **Model group**
 - Player <<**Abstract**>>
 - Club
 - Forward
 - Defender
 - Goalkeeper
 - Midfielder

- Winger

3. Business group

- Manager <<**Abstract**>>
- PlayerManager
- ClubManager
- ClubAndPlayerConnection <<**Interface**>>
- ExistIDChecker <<**Interface**>>

4. Selection group

- ManagerType <<**Enum**>>
- PlayerType <<**Enum**>>
- MenuHeaderType <<**Enum**>>
- OptionProcessor <<**Enum**>>

5. Utils group

- Acceptable/Validation <<**Interface**>>
- ComparatorContainer <<**Interface**>>
- FileIOHandler
- Inputter
- MenuContainer
- ViewHandler

3.3. Pattern Recognition

3.3.1 Pattern rule

Table 1: Format Patterns for Entities

Entity	Field	Pattern	Characteristics
Club	ClubID	CL-XXXX	Length of 7 characters: 2 initial letters, a hyphen, and 4 digits, XXXX in range [0001-9999]
	ClubName	[A-Z]/[0-9]	Only contain number and case-insensitive character
	SponsorBrand	[A-Z]/[0-9]	Only contain number and case-insensitive character
	Budget	Value > 0	Can be 0 or positive double number
Player	PlayerID	PXXXX	Length of 5 characters: 1 initial letter 'P', followed by 4 digits, XXXX in range [0001-9999]
	PlayerName	[A-Z]	Only contain case-insensitive character
	ShirtNumber	1<=Value<=99	Only contain integer number from 1 to 99

Solution:

- Using regex and isValid method to implement regex.
- Create an inputString method. Implement the inputString and isValid method by creating inputStringAndLoop method which will call inputString and isValid method to ensure data correction before saving data.
- Create multiple method inputDouble and inputInteger combine with isValid method (Use Overloading for inputDouble and inputInteger).

3.3.2 Business layer similarities

Behavior	Club Manager	Player Manager
Add	✓	✓
Search	✓	✓
Update	✓	✓
Remove	×	✓
Check exist ID	✓	✓
Get read-only data	✓	✓
Check save status	✓	✓

3.3.3 Design pattern implementation**1. Singleton pattern**

The `MenuContainer` class applies the Singleton pattern to ensure only one instance exists throughout the program. The constructor is declared `private`, and a static field `menuContainerInstance` holds the single instance. Access is provided through the static method `getInstance()`, preventing unnecessary object creation and ensuring menu state consistency across the application.

2. Template Method pattern

The abstract class `Manager<E>` defines a template for all data management operations. Common behaviors such as `add()`, `update()`, `remove()`, `search()`, and `containId()` are implemented once in the base class, while `saveData()`, `loadData()`, `show()`, and `getPathFile()` are declared abstract and implemented by `ClubManager` and `PlayerManager`. This avoids code duplication and enforces a consistent structure across all managers.

3. Static Factory pattern

Both `Club` and `Player` expose static factory methods — `Club.createNewClub()` and `Player.getNewPlayer(PlayerType)` — instead of exposing constructors directly. This provides a clear, readable way to instantiate objects and allows the creation logic to be centralized. `Player.getNewPlayer()` additionally delegates to

the `PlayerType` enum's `createPlayer()` method, combining Static Factory with Factory Method.

4. Strategy pattern

The `ComparatorContainer` interface defines multiple interchangeable `Comparator` implementations, including sorting players by club name and shirt number, and sorting clubs or players by ID. These strategies are passed into the overloaded `sortByComparator(Comparator)` method in `ClubManager` and `PlayerManager`, allowing the sorting algorithm to be swapped at runtime without modifying the manager classes.

3.4. Abstraction

Abstraction in this project means identifying the **essential attributes and behaviours** of each entity while hiding unnecessary implementation details. The result is a clean class hierarchy that models domain data faithfully and keeps each module focused on a single responsibility.

3.4.1 Data Abstraction – Entity Models

Two core domain entities are identified: `Club` and `Player`. Only the attributes required by the business logic are retained; low-level storage details (file paths, raw strings) are hidden inside utility classes.

Table 2: Essential attributes extracted for each entity

Entity	Field	Type	Role / Constraint
Club	clubId	String	Unique key; format CL-XXXX
	clubName	String	Non-empty; title-cased on set
	sponsorBrand	String	Non-empty; title-cased on set
	budget	double	Must be > 0 (million EUR)
	dataOfShirtNumber	TreeMap<Integer,String>	Maps shirt number → playerID within this club
Player	playerId	String	Unique key; format PXXXX
	clubId	String	Foreign key – must exist in ClubManager
	playerName	String	Non-empty; letters and spaces only
	position	String	Returned by polymorphic getPosition()
	shirtNumber	int	Integer 1–99; unique within same club

The Club class also exposes shirt-number management methods (`addShirtNumber`, `updateShirtNumber`, `deleteShirtNumber`, `isContainShirtNumber`) so that the integrity constraint “*shirt number unique within a club*” is enforced at the model level, not scattered across the controller.

3.4.2 Functional Abstraction – Generic Manager

All list-management behaviour shared by `ClubManager` and `PlayerManager` is abstracted into the generic base class `Manager<E>`:

Listing 1: Manager.java – abstract generic manager

```

1 public abstract class Manager<E> implements ExistIDChecker {
2     private HashMap<String, E> dataManager = new HashMap<>();
3     private boolean saveStatus;
4
5     public boolean add(String id, E data) { ... }
6     public boolean update(String id, E data) { ... }
7     public boolean remove(String id) { ... }
8     public E search(String id) { ... }

```

```

9     public boolean containId(String id) { ... }
10
11     // Abstract -- subclass provides domain-specific logic
12     public abstract void show();
13     public abstract boolean saveData();
14     public abstract boolean loadData();
15     public abstract String getPathFile();
16 }

```

By abstracting CRUD into `Manager<E>`, both concrete managers (`ClubManager`, `PlayerManager`) only need to implement the four abstract methods, removing duplicate code.

3.4.3 OOP Abstraction – Player Hierarchy (Polymorphism)

The five allowed positions are modelled as concrete subclasses of the abstract `Player` class. Position-specific behaviour is expressed through the single abstract method `getPosition()`.

Listing 2: `Player.java` – abstract base for all positions

```

1 public abstract class Player {
2     // shared fields: playerId, playerName, clubId, shirtNumber
3     public abstract String getPosition(); // overridden by each subclass
4     // ...
5 }
6
7 public class Forward extends Player {
8     public String getPosition(){
9         return "Forward";
10    }
11 }
12 public class Defender extends Player {
13     public String getPosition(){
14         return "Defender";
15    }
16 }
17 public class MidFielder extends Player{
18     public String getPosition(){
19         return "Midfielder";
20    }
21 }
22 public class Winger extends Player {
23     public String getPosition(){
24         return "Winger";
25    }
26 }
27 public class GoalKeeper extends Player{
28     public String getPosition(){
29         return "GoalKeeper";
30    }
31 }

```

The factory method `Player.getNewPlayer(PlayerType)` combined with the `PlayerType`

enum abstracts object creation so the controller never calls `new Forward()` directly:

Listing 3: Factory method + enum – abstract player creation

```

1 public enum PlayerType {
2     FORWARD("Forward") {
3         public Player createPlayer() {
4             return new Forward();
5         }
6     },
7     DEFENDER("Defender") {
8         public Player createPlayer() {
9             return new Defender();
10        }
11    }, // ... other types
12 }
13
14 // Usage in controller:
15 Player player = Player.getNewPlayer(playerType); // no concrete class name
    exposed

```

3.4.4 Interface Abstraction – Cross-Module Communication

Two interfaces decouple modules that would otherwise create circular dependencies:

Table 3: Interfaces used for abstraction

Interface	Implemented by	Purpose
ClubAndPlayerConnection	ClubManager	Exposes only the shirt-number and club-name operations that Player/PlayerManager need from ClubManager, without exposing the full ClubManager API.
ExistIDChecker	Manager<E>	Provides a single <code>containId(String)</code> method used by <code>Acceptable</code> for cross-entity ID existence checks.

3.4.5 Utility Abstraction

Low-level cross-cutting concerns are isolated into dedicated utility types:

3.5. Algorithm Design

This section specifies the step-by-step processing logic (*operation workflow*) for each major function before its Java implementation. Each algorithm follows the template: **INPUT** → **VALIDATE** → **PROCESS** → **OUTPUT**.

Table 4: Utility classes / interfaces and their abstracted responsibilities

Class / Interface	Abstracted responsibility
Acceptable	All validation logic: regex constants, isValid(), isDigitInRange(), isPartialEqual(), existence checks.
Inputter	All console input with built-in validation loops and retry/abort prompts.
ViewHandler	All console output: table formatting, line breaks, menus, name formatting.
FileIOHandler	All file I/O: readStringFile(), writeStringFile(), error logging.
ComparatorContainer	Reusable Comparator instances for sorting Club and Player lists.
MenuContainer	Singleton menu builder; decouples menu definitions from display logic.

3.5.1 Load Data From Files (startup & function 13)

Listing 4: Algorithm: Load data from files

```

1 FUNCTION loadData():
2     INPUT : clubs.txt, players.txt (from disk)
3     OUTPUT : Success message | "Load data failed!"
4
5 BEGIN
6     // Step 1: Load clubs
7     rawLines <- FileIOHandler.readStringFile("data/clubs.txt")
8     IF rawLines is empty THEN
9         PRINT "Load data failed!"
10        RETURN
11    END IF
12
13    FOR EACH line IN rawLines DO
14        tokens <- split(line, ",")
15        IF any token is empty THEN
16            clubManager.clear()
17            PRINT "Load data failed!"
18            RETURN
19        END IF
20        clubId      <- tokens[0].trim()
21        clubName    <- tokens[1].trim()
22        sponsor     <- tokens[2].trim()
23        budget      <- parseDouble(tokens[3])
24        clubManager.add(clubId, new Club(clubId, clubName, sponsor, budget))
25    END FOR
26
27    // Step 2: Load players (depends on clubs being loaded first)

```



```

28     rawLines <- FileIOHandler.readStringFile("data/players.txt")
29     IF rawLines is empty THEN
30         PRINT "Load data failed!"
31         RETURN
32     END IF
33
34     FOR EACH line IN rawLines DO
35         tokens <- split(line, ",")
36         IF any token is empty THEN
37             playerManager.clear()
38             PRINT "Load data failed!"
39             RETURN
40         END IF
41         playerId <- tokens[0].trim()
42         playerClubId<- tokens[1].trim()
43         playerName <- tokens[2].trim()
44         position <- PlayerType.searchPlayerType(tokens[3].trim())
45         shirtNumber <- parseInt(tokens[4].trim())
46         player <- Player.getNewPlayer(position)
47             .setApiClubManager(clubManager)
48             .setPlayerId(playerId)
49             .setPlayerName(playerName)
50             .setClubId(playerClubId)
51             .setShirtNumber(shirtNumber)
52         playerManager.add(playerId, player)
53     END FOR
54
55     PRINT "Load data successfully!"
56 END

```

3.5.2 Add a New Club (function 2)

Listing 5: Algorithm: Add a new club

```

1 FUNCTION addClub():
2     INPUT : clubId, clubName, sponsorBrand, budget (from console)
3     OUTPUT : "Added club successfully" | error message
4
5 BEGIN
6     // Step 1 -- Input & validate Club ID
7     clubId <- inputStringAndLoop("Input Club ID", CLUB_ID_PATTERN)
8     IF clubId is null THEN RETURN // user aborted
9     IF clubManager.containsId(clubId) THEN
10         PRINT "This club ID already exists!"
11         RETURN
12     END IF
13
14     // Step 2 -- Input & validate Club Name
15     clubName <- inputStringAndLoop("Input Club Name", CLUB_NAME_PATTERN)
16     IF clubName is null THEN RETURN

```

```

17
18 // Step 3 -- Input & validate Sponsor Brand
19 sponsor <- inputStringAndLoop("Input Sponsor Brand", SPONSOR_BRAND_PATTERN)
20 IF sponsor is null THEN RETURN
21
22 // Step 4 -- Input & validate Budget
23 budget <- inputDouble("Input budget", min = 1)
24 IF budget == 0 THEN RETURN // user chose to cancel
25
26 // Step 5 -- Create and add
27 newClub <- Club.createNewClub()
28             .setClubId(clubId)
29             .setClubName(clubName)
30             .setSponsorBrand(sponsor)
31             .setBudget(budget)
32 success <- clubManager.add(clubId, newClub)
33
34 // Step 6 -- Notify
35 IF success THEN
36     PRINT "Added club successfully"
37 ELSE
38     PRINT "Added club unsuccessfully"
39 END IF
40 END

```

3.5.3 Search for a Club by ID (function 3)

Listing 6: Algorithm: Search club by ID

```

1 FUNCTION searchClub():
2     INPUT : clubId (from console)
3     OUTPUT : Club details | "This club does not exist!"
4
5 BEGIN
6     clubId <- inputString("Input id to search: ")
7     IF clubId is null THEN RETURN
8
9     foundClub <- clubManager.search(clubId)
10
11     IF foundClub is null THEN
12         PRINT "This club does not exist!"
13     ELSE
14         PRINT foundClub.toString()
15     END IF
16 END

```

3.5.4 Update a Club by ID (function 4)

Listing 7: Algorithm: Update club by ID

```

1 FUNCTION updateClub():
2     INPUT : clubId; optional new name, sponsor, budget
3     OUTPUT : Updated club data saved | unchanged on cancel
4
5 BEGIN
6     clubId <- inputStringAndLoop("Input club id", CLUB_ID_PATTERN)
7     IF clubId is null THEN RETURN
8
9     foundClub <- clubManager.search(clubId)
10    IF foundClub is null THEN
11        PRINT "This club does not exist!"
12        RETURN
13    END IF
14
15    // Work on a temporary copy until user confirms
16    tempClub <- copy(foundClub)
17
18    LOOP
19        PRINT tempClub.toString()
20        displayUpdateClubMenu()
21        choice <- inputInteger(0..5)
22
23        SWITCH choice
24            CASE 0: // Confirm
25                clubManager.update(tempClub.getClubId(), tempClub)
26                RETURN
27            CASE 1: // Update name
28                newName <- inputStringAndLoopForUpdate(CLUB_NAME_PATTERN)
29                IF newName != null THEN tempClub.setClubName(newName)
30            CASE 2: // Update sponsor brand
31                newSponsor <- inputStringAndLoopForUpdate(SPONSOR_BRAND_PATTERN)
32                IF newSponsor != null THEN tempClub.setSponsorBrand(newSponsor)
33            CASE 3: // Update budget
34                newBudget <- inputDouble(min = 0)
35                IF newBudget != -1 THEN tempClub.setBudget(newBudget)
36            CASE 4: // Discard and exit
37                RETURN
38            CASE 5: // Change to a different club ID
39                BREAK inner loop -> restart outer loop
40        END SWITCH
41    END LOOP
42 END

```

3.5.5 Filter Clubs by Budget (function 5)

Listing 8: Algorithm: List clubs with budget $\leq$ input

```

1 FUNCTION filterClubsByBudget():
2     INPUT : maxBudget (double, from console)
3     OUTPUT : Table of clubs whose budget  $\leq$  maxBudget

```

```

4
5 BEGIN
6     PRINT "Press -1 to return!"
7     maxBudget <- inputDouble("Input budget", min = 0)
8     IF maxBudget == -1 THEN RETURN
9
10    result <- empty list
11    FOR EACH club IN clubManager.sortByComparator(ID_ASCENDING) DO
12        IF club.getBudget() <= maxBudget THEN
13            result.add(club)
14        END IF
15    END FOR
16
17    IF result is empty THEN
18        PRINT "No clubs match the condition."
19    ELSE
20        PRINT table header
21        FOR EACH club IN result DO
22            PRINT club row
23        END FOR
24    END IF
25 END

```

3.5.6 List Players Sorted by Club Name then Shirt Number (function 6)

Listing 9: Algorithm: List players sorted by club name, then shirt number

```

1 FUNCTION listPlayersByClubAndShirt():
2     INPUT : (none)
3     OUTPUT : Sorted player table
4
5 BEGIN
6     sortedPlayers <-
7         playerManager.sortByComparator(clubNameShirtNumberAscending)
8     // Comparator logic:
9     // IF clubName(A) != clubName(B) -> sort lexicographically by clubName
10    // ELSE -> sort by shirtNumber ascending
11
12    IF sortedPlayers is empty THEN
13        PRINT "No player data available."
14    ELSE
15        PRINT table header
16        FOR EACH player IN sortedPlayers DO
17            PRINT player row (resolves clubName via ClubAndPlayerConnection)
18        END FOR
19    END IF
20 END

```

3.5.7 Search Players by Partial Name (function 7)

Listing 10: Algorithm: Search players by partial name (case-insensitive)

```

1 FUNCTION searchPlayersByName():
2     INPUT : partial name keyword (>= 2 letters)
3     OUTPUT : Table of matching players
4
5 BEGIN
6     keyword <- inputString("Input partial or full name: ")
7     IF keyword is null THEN RETURN
8
9     result <- empty list
10    FOR EACH player IN playerManager.sortByComparator(ID_ASCENDING) DO
11        IF player.getPlayerName().toLowerCase() CONTAINS keyword.toLowerCase()
12            OR keyword.toLowerCase() CONTAINS player.getPlayerName().toLowerCase()
13        THEN
14            result.add(player)
15        END IF
16    END FOR
17
18    show(result)
19 END

```

3.5.8 Add a New Player (function 8)

Listing 11: Algorithm: Add a new player

```

1 FUNCTION addPlayer():
2     INPUT : playerId, name, clubId, shirtNumber, position
3     OUTPUT : "Add player successfully!" | specific error message
4
5 BEGIN
6     // Step 1 -- Player ID
7     playerId <- inputStringAndLoop("Input ID", PLAYER_ID_PATTERN)
8     IF playerId is null THEN RETURN
9     IF playerManager.containId(playerId) THEN
10        PRINT "This player ID already exists!"
11        RETURN
12    END IF
13
14    // Step 2 -- Player Name
15    name <- inputStringAndLoop("Input player name", PLAYER_NAME_PATTERN)
16    IF name is null THEN RETURN
17
18    // Step 3 -- Club ID (must exist)
19    clubId <- inputStringAndLoop("Input club id", CLUB_ID_PATTERN)
20    IF clubId is null THEN RETURN
21    IF NOT clubManager.containId(clubId) THEN
22        PRINT "This club does not exist!"
23        RETURN

```

```

24     END IF
25
26     // Step 4 -- Shirt number (unique within club, 1-99)
27     shirtNumber <- inputInteger("Input shirt number (1-99)", 1, 99)
28     IF shirtNumber == 0 THEN RETURN // user chose to cancel
29     IF clubManager.isContainShirtNumber(clubId, shirtNumber) THEN
30         PRINT "This shirt number already exists in this club!"
31         RETURN
32     END IF
33
34     // Step 5 -- Position
35     playerType <- inputPlayerType("Input player type")
36     IF playerType is null THEN RETURN
37
38     // Step 6 -- Create player; shirt number is registered in Club via API
39     player <- Player.getNewPlayer(playerType)
40         .setApiClubManager(clubManager)
41         .setPlayerId(playerId)
42         .setPlayerName(name)
43         .setClubId(clubId)
44         .setShirtNumber(shirtNumber) // internally calls
45             clubManager.addShirtNumber()
46
47     playerManager.add(playerId, player)
48     PRINT "Add player successfully!"
49 END

```

3.5.9 Remove a Player by ID (function 9)

Listing 12: Algorithm: Remove player by ID

```

1 FUNCTION removePlayer():
2     INPUT : playerId (from console)
3     OUTPUT : "Deleted player successfully!" | "This player does not exist!"
4
5 BEGIN
6     playerId <- inputStringAndLoop("Input id to delete", PLAYER_ID_PATTERN)
7     IF playerId is null THEN RETURN
8
9     player <- playerManager.search(playerId)
10    IF player is null THEN
11        PRINT "This player does not exist!"
12        RETURN
13    END IF
14
15    PRINT player.toString()
16    displayYesNoMenu()
17    choice <- inputInteger(0 = Yes delete, 1 = No cancel)
18
19    IF choice == 0 THEN

```

```

20      // Must release shirt number from club before removing player record
21      released <- player.getApiClubManager()
22                .deleteShirtNumber(player.getClubId(),
23                                   player.getShirtNumber())
24      IF released THEN
25          playerManager.remove(playerId)
26          PRINT "Deleted player successfully!"
27      ELSE
28          PRINT "Cannot delete shirt number in player club!"
29      END IF
30  ELSE
31      PRINT "You denied to delete this player!"
32  END IF
33  END

```

3.5.10 Update a Player by ID (function 10)

Listing 13: Algorithm: Update player by ID

```

1  FUNCTION updatePlayer():
2      INPUT  : playerId; optional new name, position, shirt number
3      OUTPUT : Updated player record
4
5  BEGIN
6      LOOP
7          playerId <- inputStringAndLoop("Input player id to search",
8                                         PLAYER_ID_PATTERN)
9          IF playerId is null THEN RETURN
10
11         player <- playerManager.search(playerId)
12         IF player is null THEN
13             PRINT "This player does not exist!"
14             RETURN
15         END IF
16
17         displayUpdatePlayerMenu()
18         choice <- inputInteger(0..4)
19
20         SWITCH choice
21             CASE 0: // Return to main menu
22                 RETURN
23             CASE 1: // Update player name
24                 newName <- inputStringAndLoop(PLAYER_NAME_PATTERN)
25                 IF newName != null THEN player.setPlayerName(newName)
26             CASE 2: // Update position (requires re-creating the polymorphic
27                     object)
28                 newType <- inputPlayerType()
29                 IF newType != null THEN
30                     // Preserve shirt number; rebuild player as new subclass
31                     instance

```

```

29         updatedPlayer <- Player.getNewPlayer(newType)
30             .setApiClubManager(player.getApiClubManager())
31             .setPlayerId(player.getPlayerId())
32             .setPlayerName(player.getPlayerName())
33             .setClubId(player.getClubId())
34             .setShirtNumber(player.getShirtNumber())
35         player <- updatedPlayer
36     END IF
37 CASE 3: // Update shirt number
38     newNum <- inputInteger(1, 99)
39     IF clubManager.isContainShirtNumber(player.getClubId(), newNum)
40         THEN
41             PRINT "This shirt number already exists in this club!"
42         ELSE
43             player.setShirtNumber(newNum) // internally updates Club map
44         END IF
45 CASE 4: // Change player to update (re-enter player ID)
46     CONTINUE outer LOOP
47 END SWITCH
48 playerManager.update(player.getPlayerId(), player)
49 END LOOP
50 END

```

3.5.11 List Players by Position (function 11)

Listing 14: Algorithm: List all players by a specific position

```

1 FUNCTION listPlayersByPosition():
2     INPUT : position string (case-insensitive)
3     OUTPUT : Table of players in that position
4
5 BEGIN
6     playerType <- inputPlayerType("Input player type")
7     IF playerType is null THEN RETURN
8
9     result <- empty list
10    FOR EACH player IN playerManager.sortByComparator(ID_ASCENDING) DO
11        IF player.getPosition().equalsIgnoreCase(playerType.getPosition()) THEN
12            result.add(player)
13        END IF
14    END FOR
15
16    show(result)
17 END

```

3.5.12 Save Data to Files (function 12)

Listing 15: Algorithm: Save data to files

```

1 FUNCTION saveData():
2     INPUT : (none -- uses in-memory lists)
3     OUTPUT : "Save successfully" written; clubs.txt and players.txt updated
4
5 BEGIN
6     // Clubs
7     sortedClubs <- clubManager.sortByComparator(ID_ASCENDING)
8     lines <- empty list
9     FOR EACH club IN sortedClubs DO
10         lines.add(club.toSaveString()) // "CL-0001, FC Barcelona, Nike, 820"
11     END FOR
12     FileIOHandler.writeStringFile("data/clubs.txt", lines)
13
14     // Players
15     sortedPlayers <- playerManager.sortByComparator(ID_ASCENDING)
16     lines <- empty list
17     FOR EACH player IN sortedPlayers DO
18         lines.add(player.toSaveString()) // "P0001, CL-0001, Lionel Messi,
19             Forward, 10"
20     END FOR
21     FileIOHandler.writeStringFile("data/players.txt", lines)
22
23     clubManager.setSaveStatus(true)
24     playerManager.setSaveStatus(true)
25     PRINT "Save successfully"
26 END

```

3.5.13 Load Data from Files (function 13)

Listing 16: Algorithm: Process LOAD_DATA menu option

```

1 FUNCTION processOption(playerManager, clubManager):
2     INPUT : playerManager -- Manager<Player> instance
3             clubManager -- Manager<Club> instance
4     OUTPUT : "Load data successfully!" printed; or error message on failure
5
6 BEGIN
7     TRY
8         isClubLoadSuccess <- clubManager.loadData()
9         IF NOT isClubLoadSuccess THEN
10             THROW IllegalArgumentException
11         END IF
12
13         isPlayerLoadSuccess <- playerManager.loadData()
14         IF NOT isPlayerLoadSuccess THEN
15             THROW IllegalArgumentException
16         END IF
17
18         PRINT "Load data successfully!"
19

```

```

20     CATCH NumberFormatException OR NullPointerException AS e
21         PRINT ERROR "Load data failed!"
22         FileIOHandler.logWriter(e + "-" + e.getMessage())
23
24     CATCH IllegalArgumentException AS ex
25         PRINT ERROR "Load data failed!"
26         FileIOHandler.logWriter(ex + "-" + ex.getMessage())
27     END TRY
28 END

```

Summary. All thirteen algorithms follow a consistent pattern: validate each input field immediately after reading it, reject early with a descriptive message on the first constraint violation, perform the state change only when all validations pass, and notify the user of the outcome. This separation of input, validation, business logic, and output keeps each function cohesive and easy to test independently.

4. OOP Design (UML)

4.1. Conceptual diagram

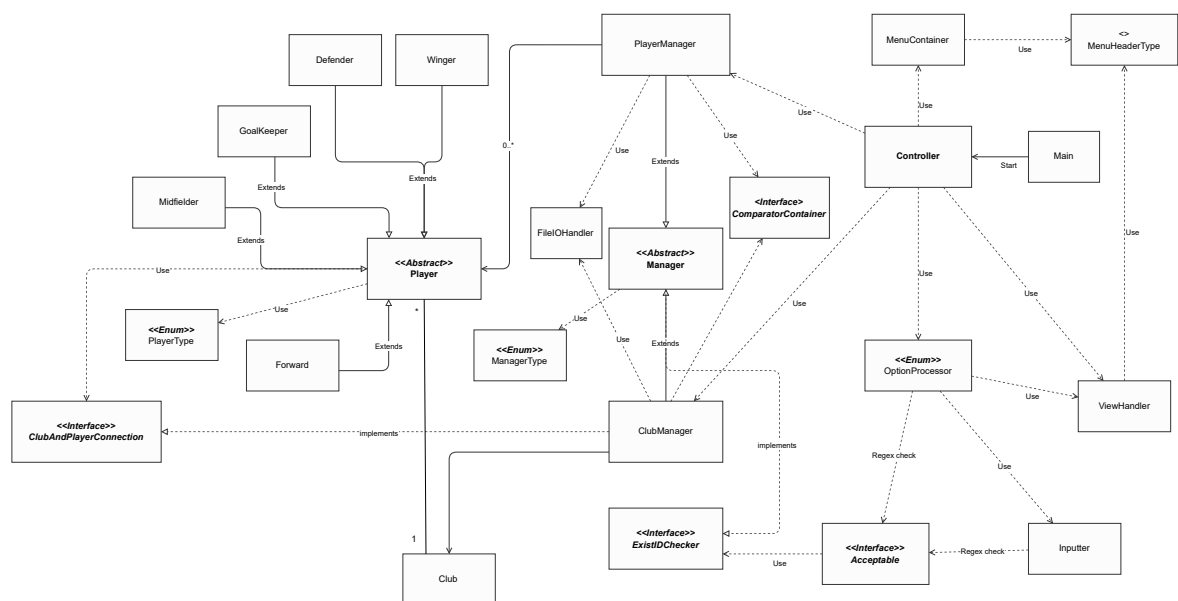


Figure 1: Conceptual diagram

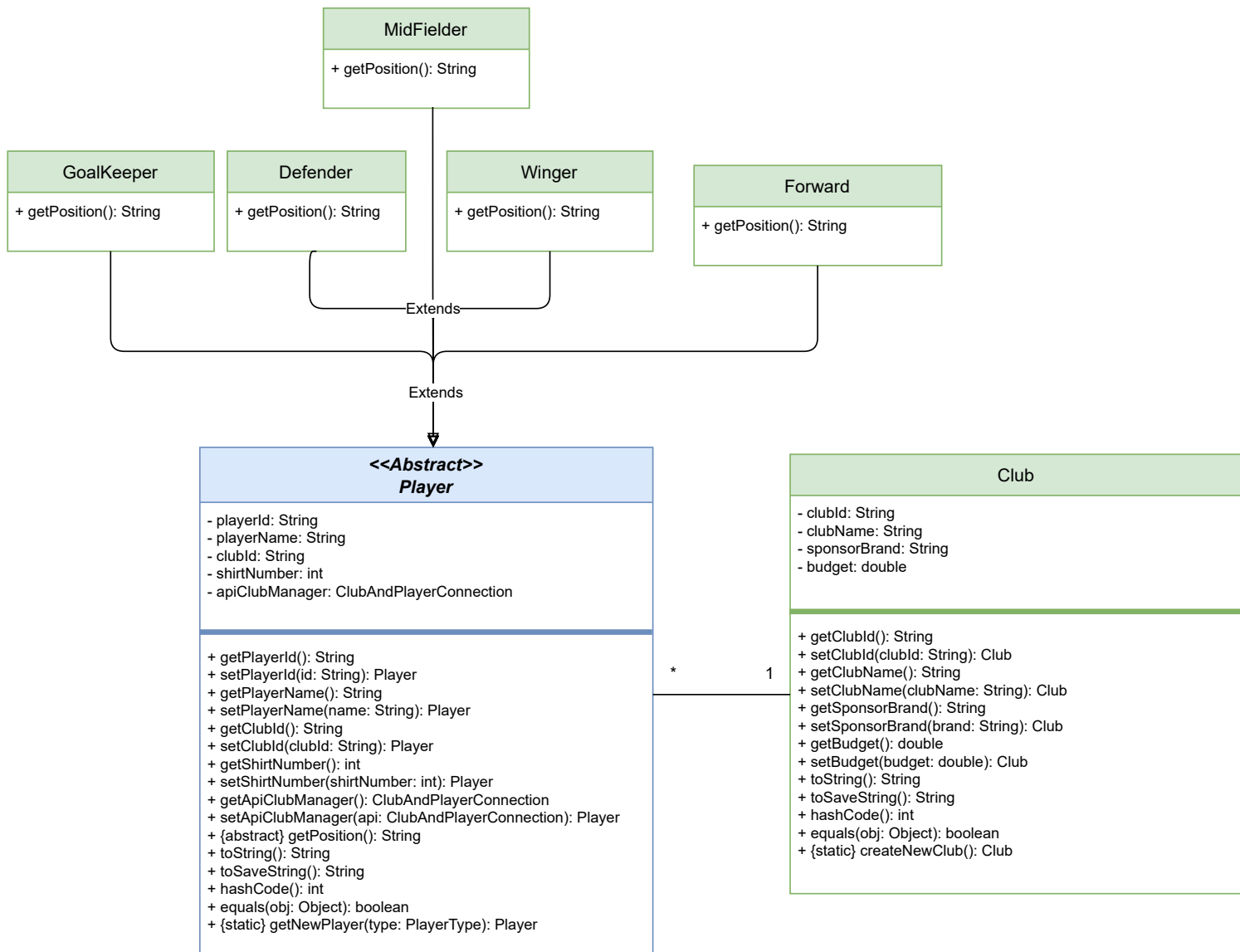
- The system is organized into five layers: **Model**, **Business**, **Utils**, **Selection**, and **Controller**.
- The **Model** layer defines the core data: an abstract `Player` class extended by five position types (`Defender`, `Winger`, `GoalKeeper`, `MidFielder`, `Forward`), and a `Club` class that tracks its players by shirt number.
- The **Business** layer manages data through an abstract `Manager` class, extended by `PlayerManager` and `ClubManager`. Two interfaces, `ClubAndPlayerConnection` and

ExistIDChecker, decouple the managers from each other and from the validation layer.

- The **Utils** layer provides shared utilities: **FileIOHandler** for file operations, **ViewHandler** for display and format, **MenuContainer** for menu building, **Inputter** for user input, and **Acceptable** for input validation.
- The **Selection** layer uses enums as factories and processors: **PlayerType** and **ManagerType** create concrete instances, while **OptionProcessor** handles each menu action.
- The **Controller** layer ties everything together. **Main** starts the program, **Controller** initializes the managers and drives the main loop by delegating each user action to **OptionProcessor**.

4.2. Class diagram decomposition

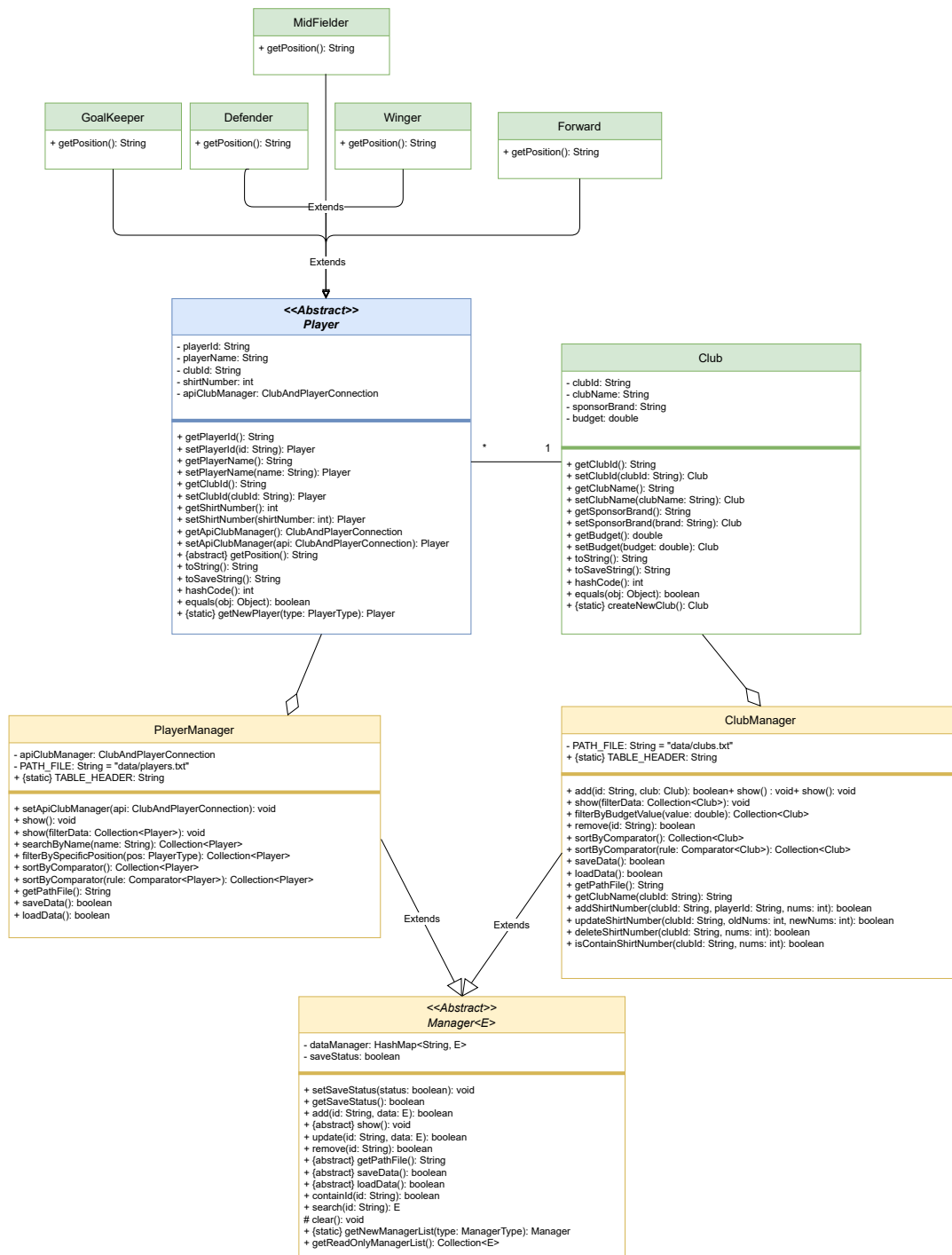
4.2.1 Player and club relationship



Description:

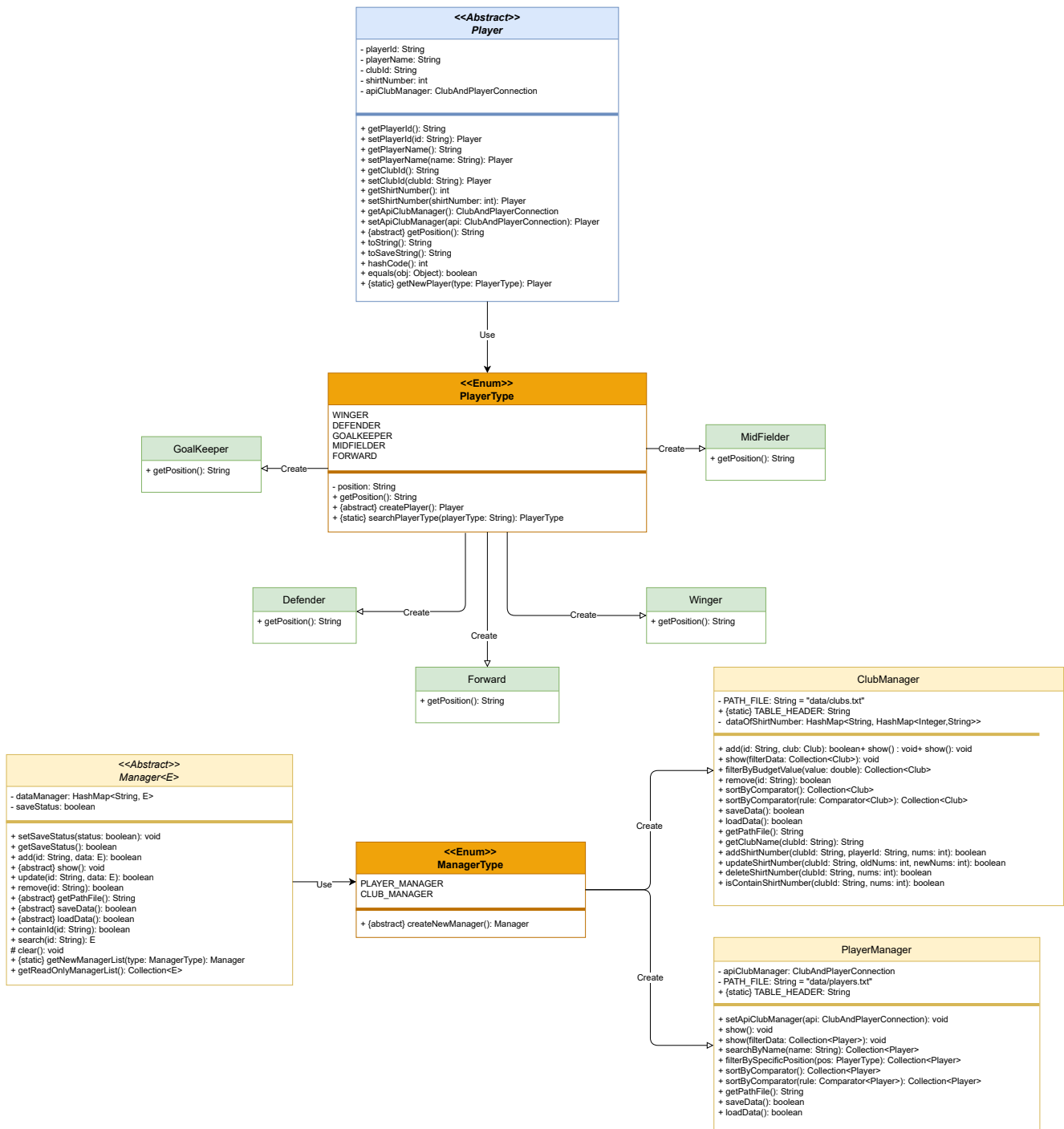
- A **Club** can contain multiple **Player** instances (1-to-many), but each **Player** belongs to exactly one **Club** at a time. **Club** internally maintains a **TreeMapInteger, String** mapping shirt numbers to player IDs, enforcing the uniqueness constraint of shirt numbers within a club at the model level rather than scattering the check across the controller.
- All of the position are players. In order to implement inheritance, **Player** will be abstract class and the other position will be its Instance which will override abstract method ***getPosition()***

4.2.2 Model and Business relationship



- `Manager<E>` is a generic abstract class that centralizes all shared CRUD operations. `Manager<E>` also serves as a Template Method foundation: common behaviors (add, update, remove, search, containId) are implemented once in the base class, except remove of `ClubManager` - it will be override in `ClubManager` due to blocking user from remove club, while `show()`, `saveData()`, `loadData()`, and `getPathFile()` are declared abstract and overridden by each concrete manager.
- `PlayerManager` stores and manages all `Player` entities, while `ClubManager` does the same for `Club` entities, both extending `Manager<E>`.

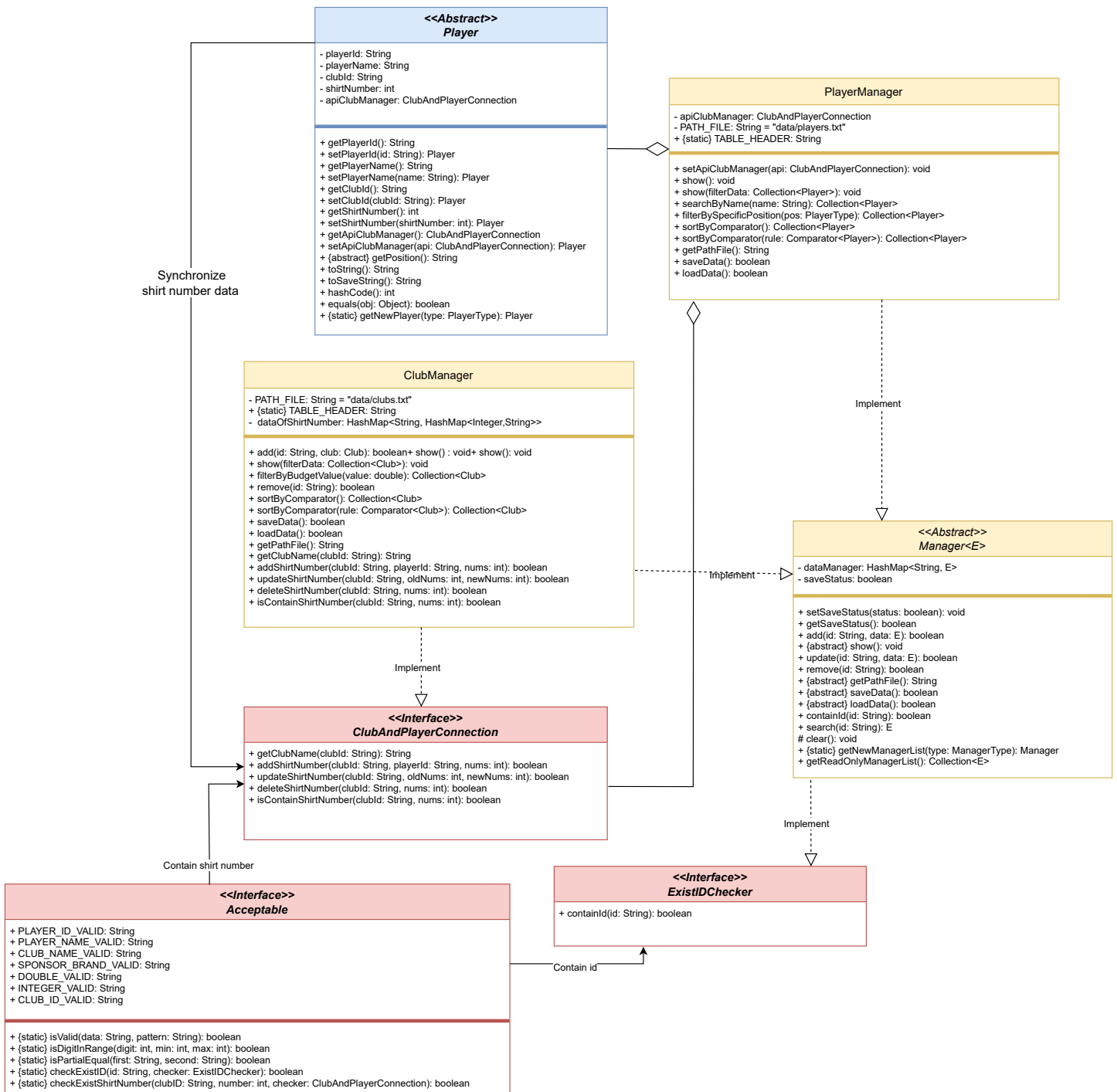
4.2.3 Enum & Static factory pattern



Description:

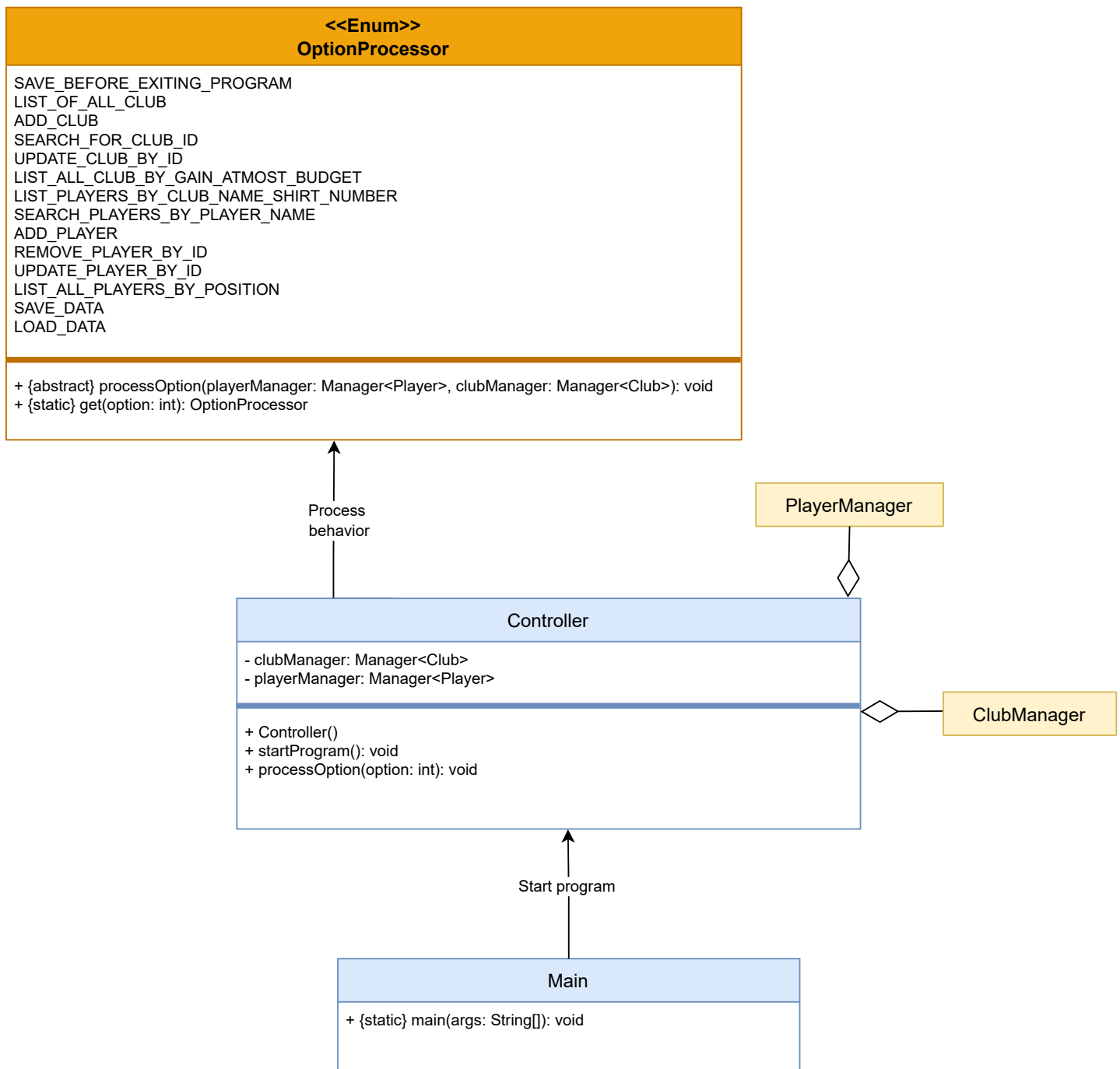
- **PlayerType** acts as a factory for **Player** subtypes: each enum constant overrides `createPlayer()` to instantiate its corresponding class, so the caller never references a concrete subclass directly. Same as **PlayerType**, **ManagerType** factories **PlayerManager** and **ClubManager** through `createNewManager()`. This pattern centralizes object creation, makes it easy to add new positions or manager types without modifying existing code, and eliminates scattered `new` calls throughout the controller.

4.2.4 Call back / Limit method access



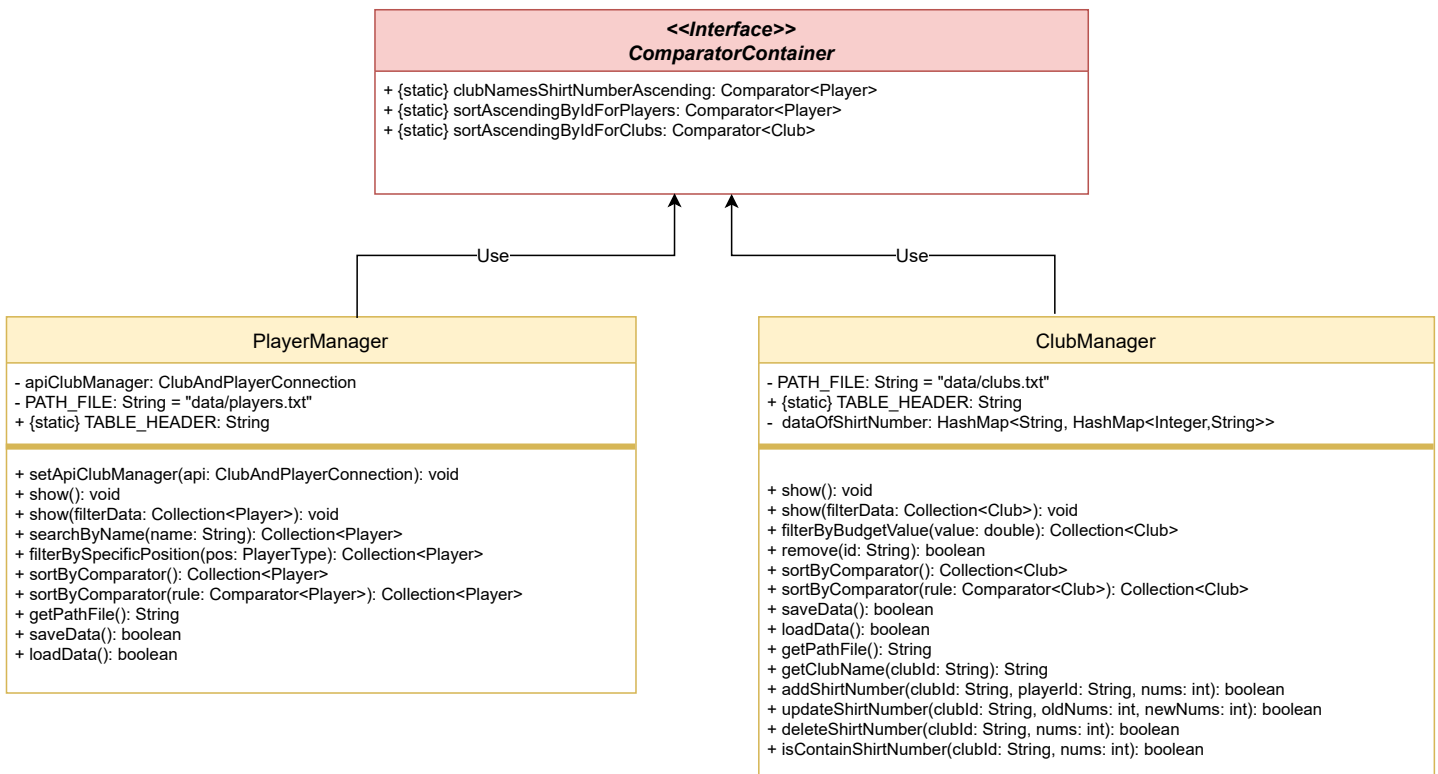
- **Player** needs to synchronize shirt number changes with its club, but directly depending on **ClubManager** would create a circular dependency. Instead, **Player** holds a reference to **ClubAndPlayerConnection** — an interface implemented by **ClubManager** — exposing only the five shirt-number operations needed. Similarly, **ExistIDChecker** exposes only **containId()** to **Acceptable**, preventing either class from accessing unrelated manager methods and enforcing the principle of least privilege across module boundaries.

4.2.5 Controller & OptionProcessor



- **Main** instantiates **Controller**, which initializes both managers and runs the main program loop. For each user input, `Controller.processOption()` delegates to `OptionProcessor.get(option)`, where each enum constant encapsulates the full logic for one menu action. This keeps **Controller** thin — it only wires components together — while **OptionProcessor** handles all business-flow decisions, making it straightforward to add or modify a menu option without touching the controller.

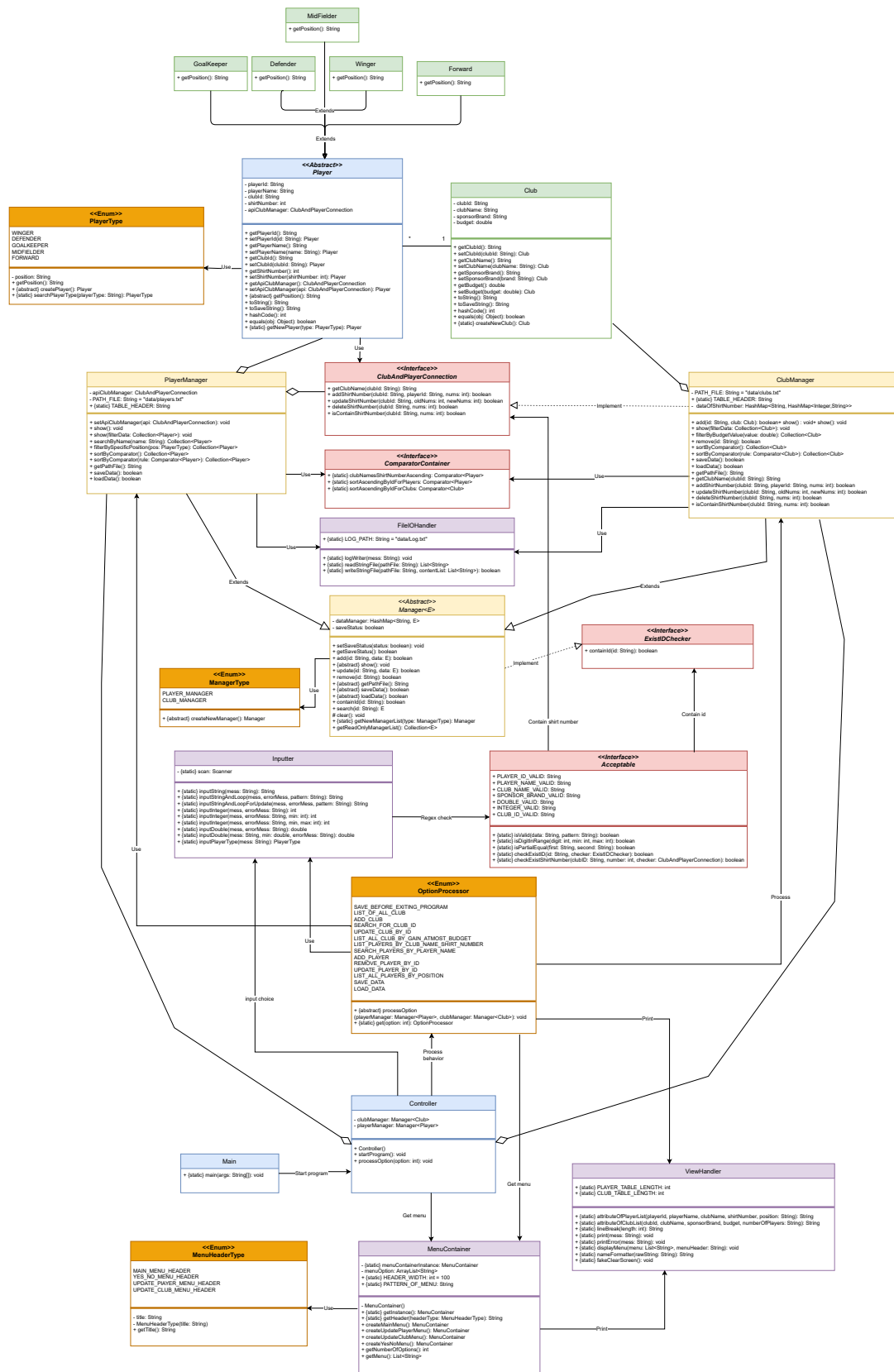
4.2.6 Business class & ComparatorContainer



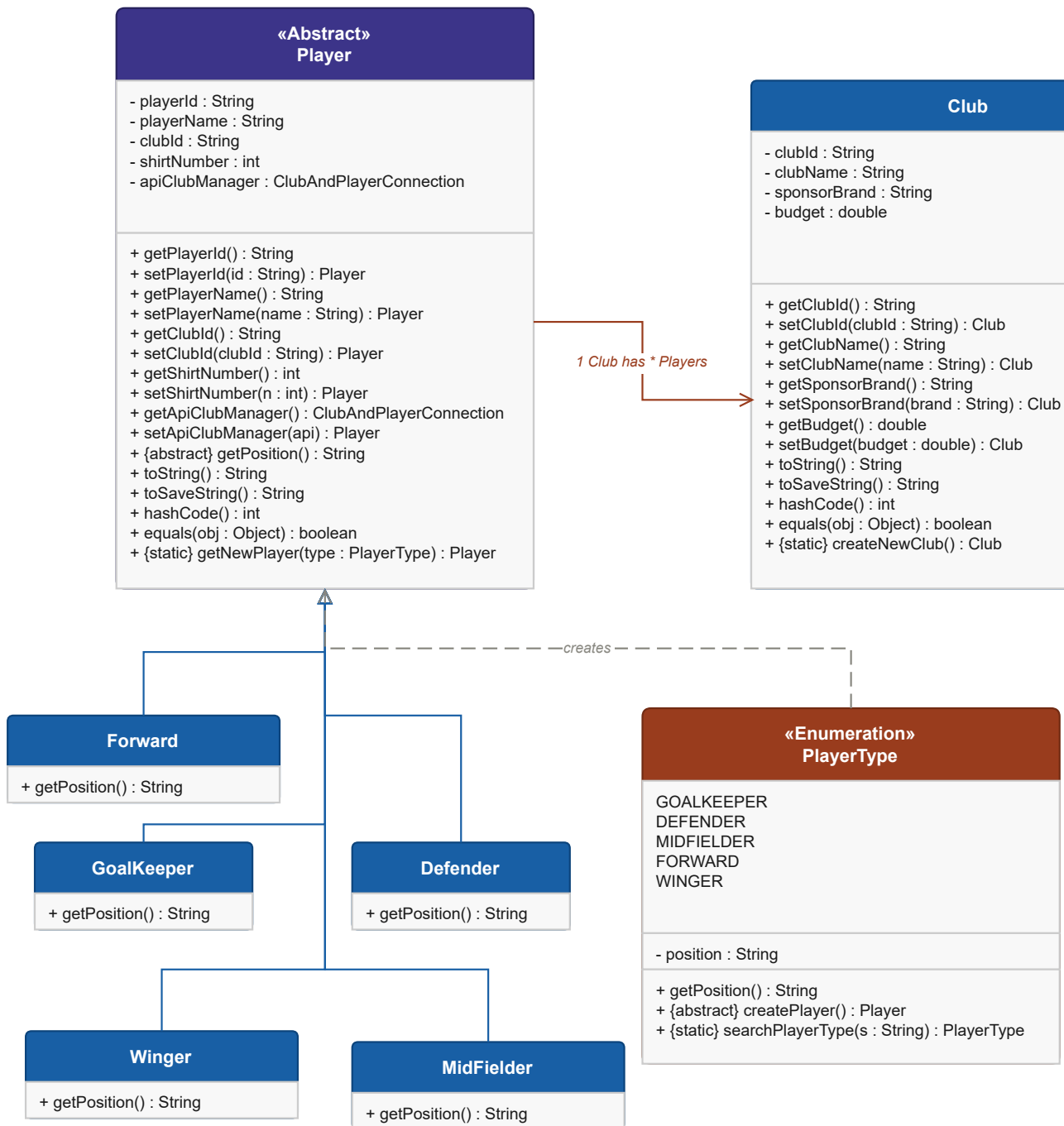
Description:

- ComparatorContainer is an interface holding three static Comparator instances: sorting players by club name then shirt number, sorting players by ID, and sorting clubs by ID. Both PlayerManager and ClubManager consume these via the overloaded sortByComparator(Comparator) method, applying the Strategy pattern to swap sorting rules at runtime without modifying the manager classes.

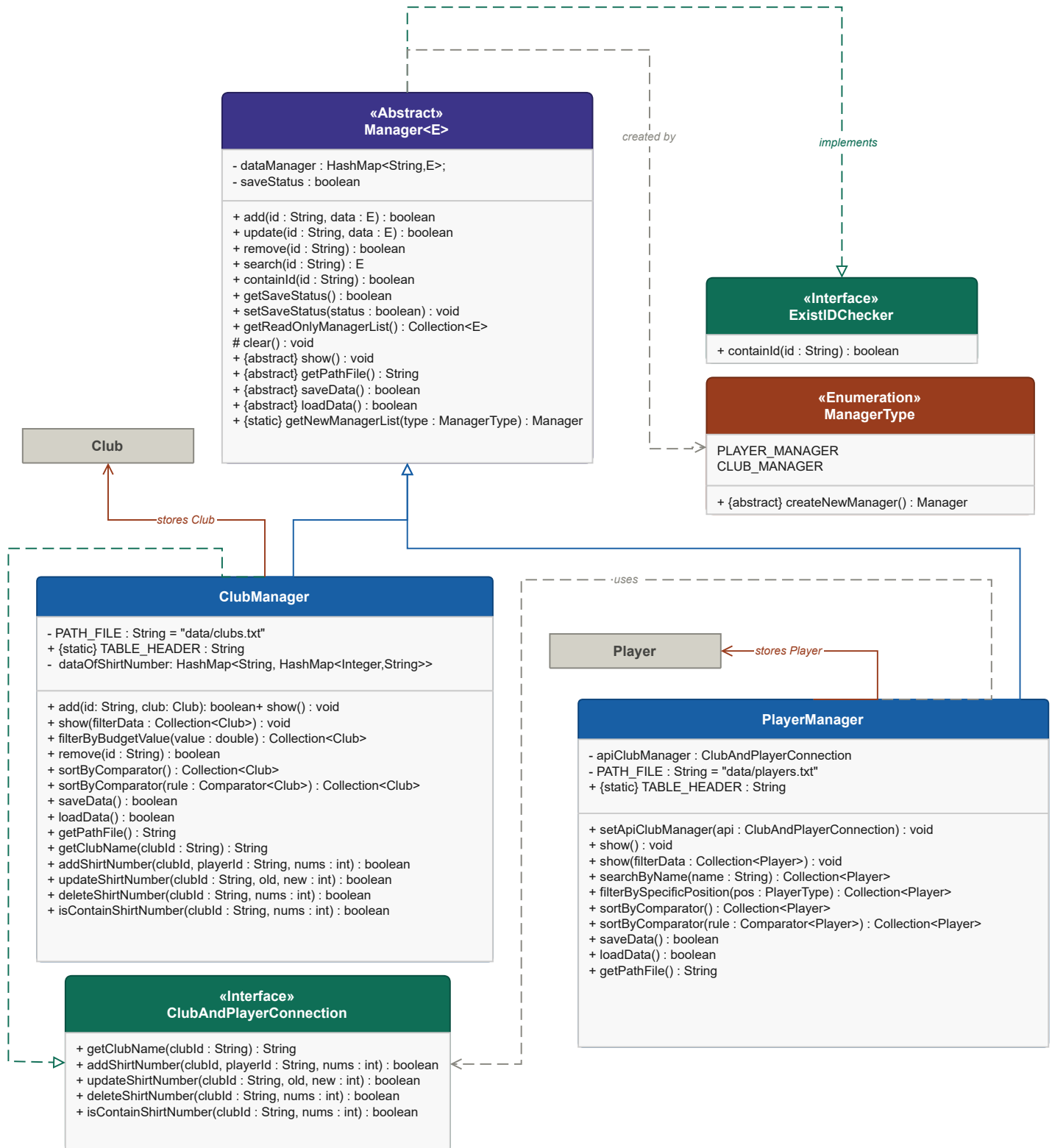
4.3. Full class diagram



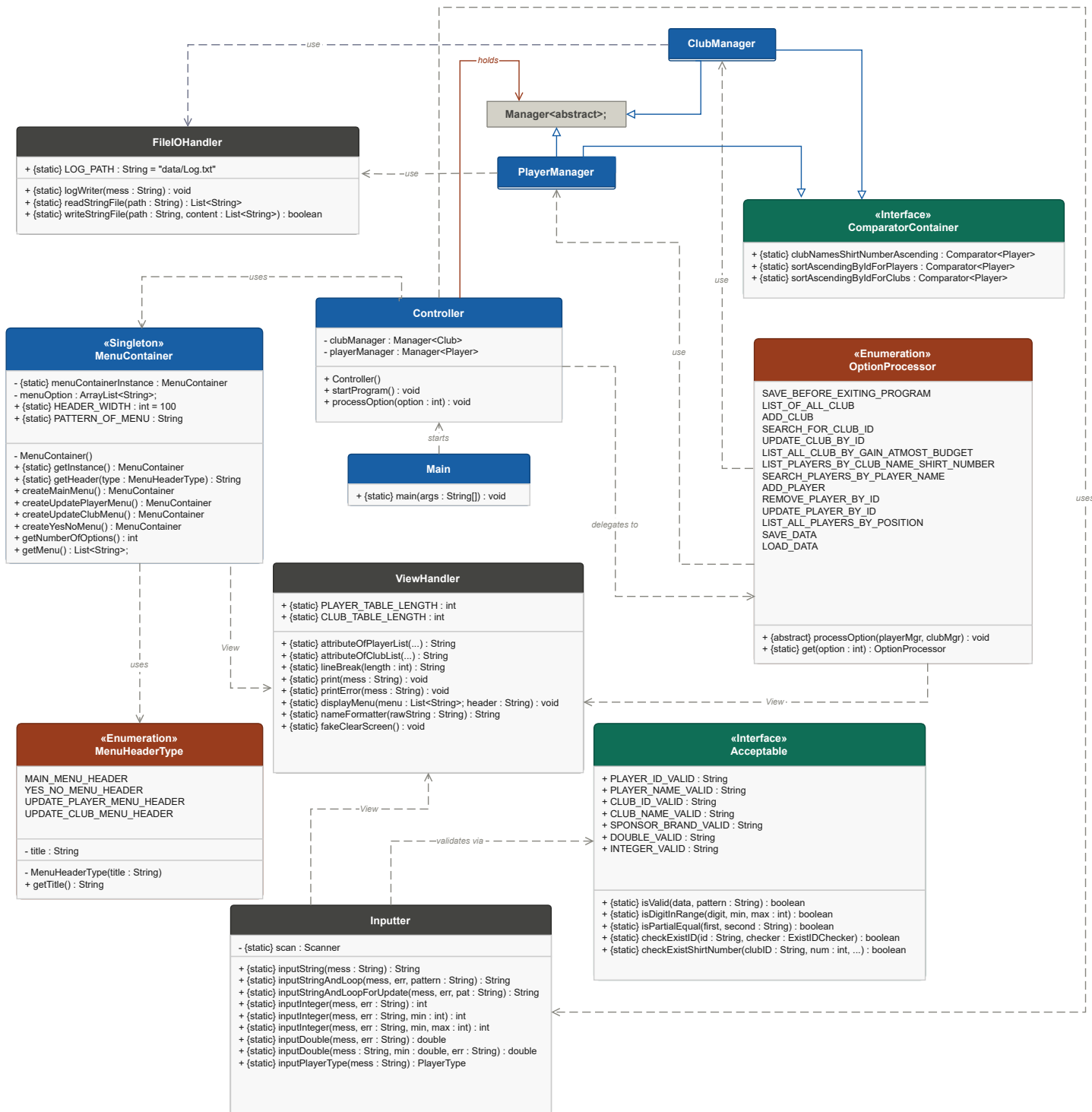
4.3.1 Model & Inheritance



4.3.2 Business layer



4.3.3 Controller & Utils



5. Conclusion

- This project successfully implements a Football Club and Player Management System using core Object-Oriented Programming principles. The layered architecture — **Model**, **Business**, **Utils**, **Selection**, and **Controller** -cleanly separates concerns and minimizes coupling between modules.
- Key design decisions include: using a generic **ManagerE** to eliminate duplicated CRUD logic; applying interface-based callbacks (**ClubAndPlayerConnection**, **ExistIDChecker**) to avoid circular dependencies; and leveraging enum-based factories (**PlayerType**, **ManagerType**) and the Strategy pattern (**ComparatorContainer**) to keep the system open for extension without modifying existing code.
- Limitations of the current system include the absence of a graphical interface, no support for concurrent data access, and a flat file storage format that does not scale well for large datasets.
- Overall, the project demonstrates that thoughtful application of Computational Thinking — Decomposition, Pattern Recognition, Abstraction, and Algorithm Design — produces a maintainable, readable, and extensible code-base that accurately models the real-world constraints of a sports management domain.

Part II

PROJECT DEMO

6. Outer function demo

7. Function 0 – Exit the Program

```
-----
| Main menu                                     |
|-----|
| 0.Exit the Program                           |
| 1.List of all clubs                         |
| 2.Add a new club                           |
| 3.Search for a club by ID                  |
| 4.Update a club by ID                     |
| 5.List of all clubs with budget  $\leq$  input value |
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |
| 7.Search players by partial player name    |
| 8.Add a new player                        |
| 9.Remove a player with ID                 |
| 10.Update a player with an ID              |
| 11.List all players by a specific position |
| 12.Save data to files                     |
| 13.Load data from files                   |
|-----|
Input your option: 0
Save successfully

Please enter to continue
```

Figure 2: Save if user did not save data

8. Function 1 – List of all clubs

8.1. Happy case

Main menu				
0.Exit the Program 1.List of all clubs 2.Add a new club 3.Search for a club by ID 4.Update a club by ID 5.List of all clubs with budget $\leq$ input value 6.List all players in ascending order of club names.if same,sort by shirt number ascending. 7.Search players by partial player name 8.Add a new player 9.Remove a player with ID 10.Update a player with an ID 11.List all players by a specific position 12.Save data to files 13.Load data from files				
Input your option: 1				
Club ID	Club name	Sponsor Brand	Budget(Million dola)	Numbers of players
CL-0001	Fc Barcelona	Nike	820.0	3
CL-0002	Real Madrid Galactico	Adidas	910.0	3
CL-0003	Manchester City Elite	Puma	880.0	2
CL-0004	Bayern Munich Prime	Adidas	860.0	2
CL-0005	Liverpool Fc	Nike	740.0	2
CL-0006	Paris Saint-germain	Nike	890.0	0
CL-0007	Juventus Fc	Adidas	700.0	0
CL-0008	As Roma Fc	Adidas	720.0	0
CL-0009	Ac Milan	Puma	610.0	0
CL-0010	Ac Milan Sportline	Puma	630.0	0
CL-0011	Chelsea Fc	Nike	760.0	0
CL-0012	Chelsea Fc X Line	Nike	780.0	3
CL-0013	Az36	Kasher	1000.0	0
CL-0015	Nike	Nike	10000.0	0
CL-0016	Aa	Nike	1000.0	0
Please enter to continue				

Figure 3: List all club

8.2. Unhappy case

```

-----
| Main menu
-----
| 0.Exit the Program
| 1.List of all clubs
| 2.Add a new club
| 3.Search for a club by ID
| 4.Update a club by ID
| 5.List of all clubs with budget ≤ input value
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending.
| 7.Search players by partial player name
| 8.Add a new player
| 9.Remove a player with ID
| 10.Update a player with an ID
| 11.List all players by a specific position
| 12.Save data to files
| 13.Load data from files
|
-----

Input your option: 1
There is no data or there is no club matches conditions to view!

Please enter to continue

```

Figure 4: When no data

9. Function 2 – Add a new club

9.1. Happy case

```

-----
| Main menu
-----
| 0.Exit the Program
| 1.List of all clubs
| 2.Add a new club
| 3.Search for a club by ID
| 4.Update a club by ID
| 5.List of all clubs with budget ≤ input value
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending.
| 7.Search players by partial player name
| 8.Add a new player
| 9.Remove a player with ID
| 10.Update a player with an ID
| 11.List all players by a specific position
| 12.Save data to files
| 13.Load data from files
|
-----

Input your option: 2
Input Club ID(Format: CL-XXXX | XXXX is from 0001-9999): cl-0015
Input Club Name: Manchester United
Input sponsor brand: Nike
Input club budget: 300
Added club successfully

Please enter to continue
|

```

Figure 5: Add club happy case

9.2. Unhappy case

```
-----
| Main menu                                     |
|-----|
| 0.Exit the Program                          |
| 1.List of all clubs                         |
| 2.Add a new club                           |
| 3.Search for a club by ID                  |
| 4.Update a club by ID                      |
| 5.List of all clubs with budget ≤ input value |
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |
| 7.Search players by partial player name    |
| 8.Add a new player                         |
| 9.Remove a player with ID                  |
| 10.Update a player with an ID              |
| 11.List all players by a specific position |
| 12.Save data to files                      |
| 13.Load data from files                    |
|-----|
Input your option: 2
Input Club ID(Format: CL-XXXX | XXXX is from 0001-9999): c1-0001

This club ID already exists!
Please enter to continue
```

Figure 6: Check duplicated id

```

-----
| Main menu                                     |
-----
| 0.Exit the Program                          |
| 1.List of all clubs                        |
| 2.Add a new club                          |
| 3.Search for a club by ID                 |
| 4.Update a club by ID                     |
| 5.List of all clubs with budget ≤ input value |
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |
| 7.Search players by partial player name   |
| 8.Add a new player                        |
| 9.Remove a player with ID                 |
| 10.Update a player with an ID              |
| 11.List all players by a specific position |
| 12.Save data to files                     |
| 13.Load data from files                   |
-----

Input your option: 2
Input Club ID(Format: CL-XXXX | XXXX is from 0001-9999): c1123455
Invalid format!
Input Club ID(Format: CL-XXXX | XXXX is from 0001-9999): abcva
Invalid format!
Input Club ID(Format: CL-XXXX | XXXX is from 0001-9999): c1-0000
Invalid format!

-----
| Choose yes or no                             |
-----
| 0.Yes                                         |
| 1.No (Can choose -1 or 1)                   |
-----

Do you want of continue?: 0
Input Club ID(Format: CL-XXXX | XXXX is from 0001-9999): c1-0015
Input Club Name: 693Havard
Please enter a valid club name!
Input Club Name: 1234
Please enter a valid club name!
Input Club Name: DongNai60Team
Input sponsor brand: 1234
Please enter a valid sponsor brand name!
Input sponsor brand: \\==
Please enter a valid sponsor brand name!
Input sponsor brand: @!\\
Please enter a valid sponsor brand name!

-----
| Choose yes or no                             |
-----
| 0.Yes                                         |
| 1.No (Can choose -1 or 1)                   |
-----

Do you want of continue?: 0
Input sponsor brand: nIKE
Input club budget: labc
Please enter valid budget or press 0 to return to menu!
Input club budget: ----
Please enter valid budget or press 0 to return to menu!
Input club budget: 1000
Added club successfully

Please enter to continue

```

Figure 7: Add club Unhappy case

10. Function 3 – Search for a club by ID

10.1. Happy case

```
-----
| Main menu
|-----
| 0.Exit the Program
| 1.List of all clubs
| 2.Add a new club
| 3.Search for a club by ID
| 4.Update a club by ID
| 5.List of all clubs with budget ≤ input value
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending.
| 7.Search players by partial player name
| 8.Add a new player
| 9.Remove a player with ID
| 10.Update a player with an ID
| 11.List all players by a specific position
| 12.Save data to files
| 13.Load data from files
|-----
Input your option: 3
Input id to search: cl-0001

-----
Club ID: CL-0001
Club name: Fc Barcelona
Sponsor Brand: Nike
Budget: 820.0
-----
```

Figure 8: Found club successfully

10.2. Unhappy case

```

-----
| Main menu                                     |
-----
| 0.Exit the Program                          |
| 1.List of all clubs                        |
| 2.Add a new club                          |
| 3.Search for a club by ID                 |
| 4.Update a club by ID                     |
| 5.List of all clubs with budget ≤ input value |
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |
| 7.Search players by partial player name    |
| 8.Add a new player                         |
| 9.Remove a player with ID                  |
| 10.Update a player with an ID              |
| 11.List all players by a specific position |
| 12.Save data to files                      |
| 13.Load data from files                    |
-----

Input your option: 3
Input id to search: a
Please enter a valid club id!
Format : CL-XXXX Where X is a number from 0 -> 9
Input id to search: cl-0000
Please enter a valid club id!
Format : CL-XXXX Where X is a number from 0 -> 9
Input id to search: cl-12

Please enter a valid club id!
Format : CL-XXXX Where X is a number from 0 -> 9
-----
| Choose yes or no                             |
-----
| 0.Yes                                         |
| 1.No (Can choose -1 or 1)                   |
-----

Do you want of continue?: 0
Input id to search: cl-0018
This club does not exist!

Please enter to continue
|

```

Figure 9: There is no club with input id

11. Function 4 – Update a club by ID

11.1. Happy case

Input your option: 4
Input club id: cl-0001

Club ID: CL-0001
Club name: Fc Barcelona
Sponsor Brand: Nike
Budget: 820.0

Update club info menu
0.Confirm information and return to menu
1.Update club name
2.Update sponsor brand
3.Update budget
4.Refuse changing information and exit
5.Confirm information and change club id to update

Input your option: 1
Enter to skip!
Input new club name: Fc Poodle
Press enter to continue!
|

Figure 10: Update club name

Club ID: CL-0001
Club name: Fc Poodle
Sponsor Brand: Nike
Budget: 820.0

Update club info menu
0.Confirm information and return to menu
1.Update club name
2.Update sponsor brand
3.Update budget
4.Refuse changing information and exit
5.Confirm information and change club id to update

Input your option: 2
Enter to skip!
Input new sponsor brand: Adiqas
Press enter to continue!
|

Figure 11: Update club sponsor brand

```
-----
Club ID: CL-0001
Club name: Fc poodle
Sponsor Brand: Adiqas
Budget: 820.0
-----

| Update club info menu |
|-----|
| 0.Confirm information and return to menu |
| 1.Update club name |
| 2.Update sponsor brand |
| 3.Update budget |
| 4.Refuse changing information and exit |
| 5.Confirm information and change club id to update |
|-----|

Input your option: 3
If you change your mind, please press 0 and press enter
Input new budget: 10000
Press enter to continue!
```

Figure 12: Update budget

```
-----
Club ID: CL-0001
Club name: Fc poodle
Sponsor Brand: Adiqas
Budget: 10000.0
-----

| Update club info menu |
|-----|
| 0.Confirm information and return to menu |
| 1.Update club name |
| 2.Update sponsor brand |
| 3.Update budget |
| 4.Refuse changing information and exit |
| 5.Confirm information and change club id to update |
|-----|

Input your option: 5
Press enter to continue!
```

Figure 13: Confirm and change club to update info

11.2. Unhappy case

```
Input club id: cl-000
This club does not exist!

Please enter to continue
|
```

Figure 14: If club is not exist

```

-----
Club ID: CL-0002
Club name: Real Madrid Galactico
Sponsor Brand: Adidas
Budget: 910.0
-----

```

```

-----
| Update club info menu |
-----
| 0.Confirm information and return to menu |
| 1.Update club name |
| 2.Update sponsor brand |
| 3.Update budget |
| 4.Refuse changing information and exit |
| 5.Confirm information and change club id to update |
-----

```

```

Input your option: 1
Input new club name:
Press enter to continue!

```

Figure 15: Enter to skip

```

-----
Club ID: CL-0002
Club name: Real Madrid Galactico
Sponsor Brand: Adidas
Budget: 910.0
-----

```

```

-----
| Update club info menu |
-----
| 0.Confirm information and return to menu |
| 1.Update club name |
| 2.Update sponsor brand |
| 3.Update budget |
| 4.Refuse changing information and exit |
| 5.Confirm information and change club id to update |
-----

```

```

Input your option: 1
Valid name: Need to contain atleast 1 character, No special character
Input new club name: @@###%%
Enter to skip!
Please enter a valid club name!
!
Input new club name: !@#%^&yt
Enter to skip!
Please enter a valid club name!
!
Input new club name:
Press enter to continue!

```

Figure 16: Check invalid new name


```
-----  
Club ID: CL-0002  
Club name: Real Madrid Galactico  
Sponsor Brand: Abc123  
Budget: 910.0  
-----
```

```
-----  
| Update club info menu  
-----  
| 0.Confirm information and return to menu  
| 1.Update club name  
| 2.Update sponsor brand  
| 3.Update budget  
| 4.Refuse changing information and exit  
| 5.Confirm information and change club id to update  
|  
-----
```

```
Input your option: 2  
Enter to skip!  
Input new sponsor brand: 1234  
Please enter a valid sponsor brand name!  
Input new sponsor brand: cl123%^$#$  
Please enter a valid sponsor brand name!  
Input new sponsor brand: -=
```

```
Please enter a valid sponsor brand name!
```

```
-----  
| Choose yes or no  
-----  
| 0.Yes  
| 1.No (Can choose -1 or 1)  
|  
-----
```

```
Do you want of continue?: 1  
Press enter to continue!
```

Figure 17: Check invalid sponsor brand

```

-----
Club ID: CL-0002
Club name: Real Madrid Galactico
Sponsor Brand: Adidas
Budget: 910.0
-----

```

```

-----
| Update club info menu                                     |
-----
| 0.Confirm information and return to menu                 |
| 1.Update club name                                       |
| 2.Update sponsor brand                                   |
| 3.Update budget                                          |
| 4.Refuse changing information and exit                   |
| 5.Confirm information and change club id to update      |
-----

```

```

Input your option: 3
If you change your mind, please press 0 and press enter
Input new budget: abc
Please enter a valid budget!
Input new budget: @!123abf&
Please enter a valid budget!
Input new budget: asv

```

```

Please enter a valid budget!

```

```

-----
| Choose yes or no                                         |
-----
| 0.Yes                                                    |
| 1.No (Can choose -1 or 1)                               |
-----

```

```

Do you want of continue?: 1
Press enter to continue!

```

Figure 18: Check invalid budget

```

-----
Club ID: CL-0001
Club name: Fc Barcelona
Sponsor Brand: Nike
Budget: 820.0
-----

```

```

-----
| Update club info menu                                     |
-----
| 0.Confirm information and return to menu                 |
| 1.Update club name                                       |
| 2.Update sponsor brand                                   |
| 3.Update budget                                          |
| 4.Refuse changing information and exit                   |
| 5.Confirm information and change club id to update      |
-----

```

```

Input your option: 4

Please enter to continue

```

Figure 19: Refuse to update

12. Function 5 – List of all clubs with budget $\leq$ input value

12.1. Happy case

Main menu				
0.Exit the Program 1.List of all clubs 2.Add a new club 3.Search for a club by ID 4.Update a club by ID 5.List of all clubs with budget $\leq$ input value 6.List all players in ascending order of club names.if same,sort by shirt number ascending. 7.Search players by partial player name 8.Add a new player 9.Remove a player with ID 10.Update a player with an ID 11.List all players by a specific position 12.Save data to files 13.Load data from files				
Input your option: 5				
Press -1 to return!				
Input budget: 1000				
Club ID	Club name	Sponsor Brand	Budget	Numbers of players
CL-0001	Fc Barcelona	Nike	820.0	3
CL-0002	Real Madrid Galactico	Adidas	910.0	3
CL-0003	Manchester City Elite	Puma	880.0	2
CL-0004	Bayern Munich Prime	Adidas	860.0	2
CL-0005	Liverpool Fc	Nike	740.0	2
CL-0006	Paris Saint-germain	Nike	890.0	0
CL-0007	Juventus Fc	Adidas	700.0	0
CL-0008	As Roma Fc	Adidas	720.0	0
CL-0009	Ac Milan	Puma	610.0	0
CL-0010	Ac Milan Sportline	Puma	630.0	0
CL-0011	Chelsea Fc	Nike	760.0	0
CL-0012	Chelsea Fc X Line	Nike	780.0	3
CL-0013	Az36	Kasher	1000.0	0
CL-0016	Aa	Nike	1000.0	0
Please enter to continue				

Figure 20: List all club with budget $\leq$ 1000

12.2. Unhappy case

```
|-----|
| Main menu                                     |
|-----|
| 0.Exit the Program                          |
| 1.List of all clubs                         |
| 2.Add a new club                           |
| 3.Search for a club by ID                  |
| 4.Update a club by ID                      |
| 5.List of all clubs with budget ≤ input value |
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |
| 7.Search players by partial player name     |
| 8.Add a new player                         |
| 9.Remove a player with ID                  |
| 10.Update a player with an ID              |
| 11.List all players by a specific position  |
| 12.Save data to files                      |
| 13.Load data from files                    |
|-----|
Input your option: 5
Press -1 to return!
Input budget: 100
There is no data or there is no club matches conditions to view!

Please enter to continue
```

Figure 21: If there is no club matches condition

13. Function 6 – List all players in ascending order of club names, if same sort by shirt number ascending

Main menu					
0.Exit the Program 1.List of all clubs 2.Add a new club 3.Search for a club by ID 4.Update a club by ID 5.List of all clubs with budget $\leq$ input value 6.List all players in ascending order of club names.if same,sort by shirt number ascending. 7.Search players by partial player name 8.Add a new player 9.Remove a player with ID 10.Update a player with an ID 11.List all players by a specific position 12.Save data to files 13.Load data from files					
Input your option: 6					
Player ID	Player Name	Club Name	Shirt Number	Position	
P0010	Joshua Kimmich	Bayern Munich Prime	6	Midfielder	
P0009	Thomas Müller	Bayern Munich Prime	25	Forward	
P0013	Raheem Sterling	Chelsea Fc X Line	7	Winger	
P0014	Enzo Fernández	Chelsea Fc X Line	8	Midfielder	
P0015	Reece James	Chelsea Fc X Line	24	Defender	
P0002	Sergio Busquets	Fc Barcelona	5	Midfielder	
P0001	Lionel Messi	Fc Barcelona	10	Forward	
P0003	Jordi Alba	Fc Barcelona	18	Defender	
P0012	Virgil Van Dijk	Liverpool Fc	4	Defender	
P0011	Mohamed Salah	Liverpool Fc	11	Forward	
P0007	Erling Haaland	Manchester City Elite	9	Forward	
P0008	Kevin De Bruyne	Manchester City Elite	17	Midfielder	
P0005	Toni Kroos	Real Madrid Galactico	8	Midfielder	
P0004	Luka Modrić	Real Madrid Galactico	10	Midfielder	
P0006	Rodrygo Silva	Real Madrid Galactico	21	Forward	
Please enter to continue					

Figure 22: List all players by club names, shirt number ascending

14. Function 7 – Search players by partial player name

14.1. Happy case

```

-----
| Main menu
-----
| 0.Exit the Program
| 1.List of all clubs
| 2.Add a new club
| 3.Search for a club by ID
| 4.Update a club by ID
| 5.List of all clubs with budget ≤ input value
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending.
| 7.Search players by partial player name
| 8.Add a new player
| 9.Remove a player with ID
| 10.Update a player with an ID
| 11.List all players by a specific position
| 12.Save data to files
| 13.Load data from files
-----

Input your option: 7
Input name (partial or full) to search: ee

-----
| Player ID | Player Name | Club Name | Shirt Number | Position |
-----
| P0013 | Raheem Sterling | Chelsea Fc X Line | 7 | Winger |
-----
| P0015 | Reece James | Chelsea Fc X Line | 24 | Defender |
-----

Please enter to continue
|

```

Figure 23: if there is atleast 1 player matches

14.2. Unhappy case

```

-----
| Main menu
-----
| 0.Exit the Program
| 1.List of all clubs
| 2.Add a new club
| 3.Search for a club by ID
| 4.Update a club by ID
| 5.List of all clubs with budget ≤ input value
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending.
| 7.Search players by partial player name
| 8.Add a new player
| 9.Remove a player with ID
| 10.Update a player with an ID
| 11.List all players by a specific position
| 12.Save data to files
| 13.Load data from files
-----

Input your option: 7
Input name (partial or full) to search: asd
There is no data or there is no player matches conditions to view!

Please enter to continue
|

```

Figure 24: If there is no player matches

15. Function 8 – Add a new player

15.1. Happy case

```

-----
| Main menu                                     |
-----
| 0.Exit the Program                          |
| 1.List of all clubs                         |
| 2.Add a new club                           |
| 3.Search for a club by ID                  |
| 4.Update a club by ID                      |
| 5.List of all clubs with budget ≤ input value |
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |
| 7.Search players by partial player name     |
| 8.Add a new player                          |
| 9.Remove a player with ID                  |
| 10.Update a player with an ID               |
| 11.List all players by a specific position  |
| 12.Save data to files                      |
| 13.Load data from files                    |
-----

Input your option: 8
Input ID(Format: PXXXX (XXXX is from 0001-9999)): P0036
Input player name: Muhammed Ali

-----
| Club ID | Club name | Sponsor Brand | Budget | Numbers of players |
-----
| CL-0001 | Fc Barcelona | Nike | 820.0 | 3 |
-----
| CL-0002 | Real Madrid Galactico | Adidas | 910.0 | 3 |
-----
| CL-0003 | Manchester City Elite | Puma | 880.0 | 2 |
-----
| CL-0004 | Bayern Munich Prime | Adidas | 860.0 | 2 |
-----
| CL-0005 | Liverpool Fc | Nike | 740.0 | 2 |
-----
| CL-0006 | Paris Saint-germain | Nike | 890.0 | 0 |
-----
| CL-0007 | Juventus Fc | Adidas | 700.0 | 0 |
-----
| CL-0008 | As Roma Fc | Adidas | 720.0 | 0 |
-----
| CL-0009 | Ac Milan | Puma | 610.0 | 0 |
-----
| CL-0010 | Ac Milan Sportline | Puma | 630.0 | 0 |
-----
| CL-0011 | Chelsea Fc | Nike | 760.0 | 0 |
-----
| CL-0012 | Chelsea Fc X Line | Nike | 780.0 | 3 |
-----
| CL-0013 | Az36 | Kasher | 1000.0 | 0 |
-----
| CL-0015 | Nike | Nike | 10000.0 | 0 |
-----
| CL-0016 | Aa | Nike | 1000.0 | 0 |
-----

Input club id(Format: CL-XXXX | XXXX is from 0001-9999): cl-0005
Input shirt number(1->99) or press 0 to return: 99
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input player type: defender
Add player successfully!

Please enter to continue

```

Figure 25: Adding player in happy case

15.2. Unhappy case

```

-----
| Main menu
-----
| 0.Exit the Program
| 1.List of all clubs
| 2.Add a new club
| 3.Search for a club by ID
| 4.Update a club by ID
| 5.List of all clubs with budget ≤ input value
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending.
| 7.Search players by partial player name
| 8.Add a new player
| 9.Remove a player with ID
| 10.Update a player with an ID
| 11.List all players by a specific position
| 12.Save data to files
| 13.Load data from files
-----

Input your option: 8

Please enter to continue
Please add club before adding new players

```

Figure 26: If there is no club data

```

-----
| Main menu
-----
| 0.Exit the Program
| 1.List of all clubs
| 2.Add a new club
| 3.Search for a club by ID
| 4.Update a club by ID
| 5.List of all clubs with budget ≤ input value
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending.
| 7.Search players by partial player name
| 8.Add a new player
| 9.Remove a player with ID
| 10.Update a player with an ID
| 11.List all players by a specific position
| 12.Save data to files
| 13.Load data from files
-----

Input your option: 8
Input ID(Format: PXXXX (XXXX is from 0001-9999)): abc
Invalid format!
Input ID(Format: PXXXX (XXXX is from 0001-9999)): p0000
Invalid format!
Input ID(Format: PXXXX (XXXX is from 0001-9999)): 0
Invalid format!

-----
| Choose yes or no
-----
| 0.Yes
| 1.No (Can choose -1 or 1)
-----

Do you want of continue?: 0
Input ID(Format: PXXXX (XXXX is from 0001-9999)): p0001

This player ID already exists!
Please enter to continue

```

Figure 27: Exist id check - Wrong ID format


```

-----
Input your option: 8
Input ID(Format: PXXXX (XXXX is from 0001-9999)): P0036
Input player name: a999
Please enter correct name!
Name can not contain numbers and special characters
Input player name: a
Please enter correct name!
Name can not contain numbers and special characters
Input player name: *****124hahas
Please enter correct name!

-----
Name can not contain numbers and special characters
| Choose yes or no |
-----
| 0.Yes |
| 1.No (Can choose -1 or 1) |
|-----|

Do you want of continue?: 0
Input player name: Mike Tyson
Club List
-----
| Club ID | Club name | Sponsor Brand | Budget(Million dola)| Numbers of players |
-----
| CL-0001 | Fc Barcelona | Nike | 820.0 | 3 |
-----
| CL-0002 | Real Madrid Galactico | Adidas | 910.0 | 3 |
-----
| CL-0003 | Manchester City Elite | Puma | 880.0 | 2 |
-----
| CL-0004 | Bayern Munich Prime | Adidas | 860.0 | 2 |
-----
| CL-0005 | Liverpool Fc | Nike | 740.0 | 2 |
-----
| CL-0006 | Paris Saint-germain | Nike | 890.0 | 0 |
-----
| CL-0007 | Juventus Fc | Adidas | 700.0 | 0 |
-----
| CL-0008 | As Roma Fc | Adidas | 720.0 | 0 |
-----
| CL-0009 | Ac Milan | Puma | 610.0 | 0 |
-----
| CL-0010 | Ac Milan Sportline | Puma | 630.0 | 0 |
-----
| CL-0011 | Chelsea Fc | Nike | 760.0 | 0 |
-----
| CL-0012 | Chelsea Fc X Line | Nike | 780.0 | 3 |
-----
| CL-0013 | Az36 | Kasher | 1000.0 | 0 |
-----
| CL-0015 | Nike | Nike | 10000.0 | 0 |
-----
| CL-0016 | Aa | Nike | 1000.0 | 0 |
-----
Input club id(Format: CL-XXXX | XXXX is from 0001-9999): clabc
Invalid club id format
Input club id(Format: CL-XXXX | XXXX is from 0001-9999): cl-0000
Invalid club id format
Input club id(Format: CL-XXXX | XXXX is from 0001-9999): cl-0001
Input shirt number(1->99) or press 0 to return: abc
Please input number!
Input shirt number(1->99) or press 0 to return: @@#
Please input number!
Input shirt number(1->99) or press 0 to return: 5

Please enter to continue
This number already exist in this club

```

Figure 28: Check exist shirt number

16. Function 9 – Remove a player with ID

16.1. Happy case

```
-----
| Main menu                                     |
|-----|
| 0.Exit the Program                          |
| 1.List of all clubs                         |
| 2.Add a new club                           |
| 3.Search for a club by ID                  |
| 4.Update a club by ID                      |
| 5.List of all clubs with budget ≤ input value |
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |
| 7.Search players by partial player name     |
| 8.Add a new player                         |
| 9.Remove a player with ID                  |
| 10.Update a player with an ID              |
| 11.List all players by a specific position  |
| 12.Save data to files                     |
| 13.Load data from files                    |
|-----|

Input your option: 9
Input id to delete: p0001

-----
Player ID: P0001
Player Name: Lionel Messi
Player's club id: CL-0001
Shirt Number: 10
Position: Forward
-----

-----
| Choose yes or no                             |
|-----|
| 0.Yes                                         |
| 1.No (Can choose -1 or 1)                   |
|-----|

Are you sure to delete this player? 0
Deleted player successfully!

Please enter to continue
```

Figure 29: Deleting a player successfully

16.2. Unhappy case

```

-----
| Main menu                                     |
-----
| 0.Exit the Program                          |
| 1.List of all clubs                        |
| 2.Add a new club                          |
| 3.Search for a club by ID                 |
| 4.Update a club by ID                     |
| 5.List of all clubs with budget ≤ input value |
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |
| 7.Search players by partial player name    |
| 8.Add a new player                        |
| 9.Remove a player with ID                 |
| 10.Update a player with an ID              |
| 11.List all players by a specific position |
| 12.Save data to files                     |
| 13.Load data from files                   |
-----
Input your option: 9
Input id to delete: p0000
This player does not exist!

Please enter to continue

```

Figure 30: If there is no id matches

17. Function 10 – Update a player with an ID

17.1. Happy case

```

-----
Player ID: P0001
Player Name: Lionel Messi
Player's club id: CL-0001
Shirt Number: 10
Position: Forward
-----

-----
| Update Player info menu                     |
-----
| 0.Return to menu                          |
| 1.Update player name                      |
| 2.Update position                        |
| 3.Update shirt number                    |
| 4.Change player id to update              |
-----
Input your option: 1
Input new name: Lion Messi
Enter to continue|

```

Figure 31: Change player name

```
-----  
Player ID: P0001  
Player Name: Lion Messi  
Player's club id: CL-0001  
Shirt Number: 10  
Position: Forward  
-----
```

```
-----  
| Update Player info menu  
-----  
| 0.Return to menu  
| 1.Update player name  
| 2.Update position  
| 3.Update shirt number  
| 4.Change player id to update  
|  
-----
```

```
Input your option: 2  
Input position: Midfielder  
Enter to continue!|
```

Figure 32: Change player position

```
-----  
Player ID: P0001  
Player Name: Lion Messi  
Player's club id: CL-0001  
Shirt Number: 10  
Position: Midfielder  
-----
```

```
-----  
| Update Player info menu  
-----  
| 0.Return to menu  
| 1.Update player name  
| 2.Update position  
| 3.Update shirt number  
| 4.Change player id to update  
|  
-----
```

```
Input your option: 3  
Input new shirt number: 11  
Enter to continue!|
```

Figure 33: Change player shirt number

```

-----
Player ID: P0001
Player Name: Lion Messi
Player's club id: CL-0001
Shirt Number: 11
Position: Midfielder
-----

```

```

-----
| Update Player info menu |
-----
| 0.Return to menu |
| 1.Update player name |
| 2.Update position |
| 3.Update shirt number |
| 4.Change player id to update |
-----

```

Input your option: 4

Please enter to continue

Figure 34: Change player to update

17.2. Unhappy case

Input player id to search: p0001

```

-----
Player ID: P0001
Player Name: Lionel Messi
Player's club id: CL-0001
Shirt Number: 10
Position: Forward
-----

```

```

-----
| Update Player info menu |
-----
| 0.Return to menu |
| 1.Update player name |
| 2.Update position |
| 3.Update shirt number |
| 4.Change player id to update |
-----

```

```

Input your option: 1
Enter to skip!
Input new name: abc2123
Invalid name
Please enter again!!
Input new name: abc*&^%
Invalid name
Please enter again!!
Input new name: 11
Invalid name
Please enter again!!

```

```

-----
| Choose yes or no |
-----
| 0.Yes |
| 1.No (Can choose -1 or 1) |
-----

```

```

Do you want of continue?: 1
Enter to continue!

```

Figure 35: Player name valid check

```

-----
Player ID: P0001
Player Name: Lionel Messi
Player's club id: CL-0001
Shirt Number: 10
Position: Forward
-----

```

```

-----
| Update Player info menu |
-----
| 0.Return to menu |
| 1.Update player name |
| 2.Update position |
| 3.Update shirt number |
| 4.Change player id to update |
-----

```

```

Input your option: 2
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input position:
Please enter a valid player type!
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input position:
Please enter a valid player type!
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input position:

```

Please enter a valid player type!

```

-----
| Choose yes or no |
-----
| 0.Yes |
| 1.No (Can choose -1 or 1) |
-----

```

```

Do you want of continue?: 0
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input position: aasd
Please enter a valid player type!
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input position: 123567fga
Please enter a valid player type!
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input position: *(
Please enter a valid player type!
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input position:

```

Please enter a valid player type!

```

-----
| Choose yes or no |
-----
| 0.Yes |
| 1.No (Can choose -1 or 1) |
-----

```

```

Do you want of continue?: 1
Enter to continue!|

```

Figure 36: Position valid check

```
-----  
Player ID: P0001  
Player Name: Lionel Messi  
Player's club id: CL-0001  
Shirt Number: 6  
Position: Forward  
-----
```

```
-----  
| Update Player info menu  
-----  
| 0.Return to menu  
| 1.Update player name  
| 2.Update position  
| 3.Update shirt number  
| 4.Change player id to update  
-----
```

```
Input your option: 3  
Input new shirt number: 5  
This shirt number already exist in this club!  
Enter to continue!|
```

Figure 37: Duplicate shirt number check

```
-----  
Player ID: P0001  
Player Name: Lionel Messi  
Player's club id: CL-0001  
Shirt Number: 10  
Position: Forward  
-----
```

```
-----  
| Update Player info menu  
-----  
| 0.Return to menu  
| 1.Update player name  
| 2.Update position  
| 3.Update shirt number  
| 4.Change player id to update  
-----
```

```
Input your option: 3  
Input new shirt number: 10  
This shirt number is this player current shirt number!  
Enter to continue!
```

Figure 38: If new shirt number = old number

```

-----
Player ID: P0001
Player Name: Lionel Messi
Player's club id: CL-0001
Shirt Number: 10
Position: Forward
-----

```

```

-----
| Update Player info menu |
-----
| 0.Return to menu       |
| 1.Update player name   |
| 2.Update position      |
| 3.Update shirt number  |
| 4.Change player id to update |
|                         |
-----

```

```

Input your option: 3
Input new shirt number: 123
Please input a valid shirt number!(1-99)
Input new shirt number: abc^^&%
Please input number!
Press 0 to skip
Input new shirt number: +_+
Please input number!
Press 0 to skip
Input new shirt number: ===
Please input number!
Press 0 to skip
Input new shirt number:
Please input number!
Press 0 to skip
Input new shirt number: 0
Enter to continue!
-----

```

Figure 39: Check valid shirt number and ignore change

```

-----
Player ID: P0001
Player Name: Lionel Messi
Player's club id: CL-0001
Shirt Number: 10
Position: Forward
-----

```

```

-----
| Update Player info menu |
-----
| 0.Return to menu       |
| 1.Update player name   |
| 2.Update position      |
| 3.Update shirt number  |
| 4.Change player id to update |
|                         |
-----

```

```

Input your option: 0

```

```

Please enter to continue

```

Figure 40: Return to main menu

18. Function 11 – List all players by a specific position

18.1. Happy case

```

|-----|
| Main menu                                     |
|-----|
| 0.Exit the Program                           |
| 1.List of all clubs                          |
| 2.Add a new club                             |
| 3.Search for a club by ID                    |
| 4.Update a club by ID                       |
| 5.List of all clubs with budget ≤ input value |
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |
| 7.Search players by partial player name      |
| 8.Add a new player                           |
| 9.Remove a player with ID                    |
| 10.Update a player with an ID                |
| 11.List all players by a specific position   |
| 12.Save data to files                       |
| 13.Load data from files                     |
|-----|
Input your option: 11
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input player type: defender

|-----|
| Player ID | Player Name | Club Name | Shirt Number | Position |
|-----|
| P0003 | Jordi Alba | Fc Barcelona | 18 | Defender |
|-----|
| P0012 | Virgil Van Dijk | Liverpool Fc | 4 | Defender |
|-----|
| P0015 | Reece James | Chelsea Fc X Line | 24 | Defender |
|-----|
Please enter to continue

```

Figure 41: List player by defender

18.2. Unhappy case

```

-----
| Main menu                                     |
-----
| 0.Exit the Program                           |
| 1.List of all clubs                         |
| 2.Add a new club                           |
| 3.Search for a club by ID                  |
| 4.Update a club by ID                     |
| 5.List of all clubs with budget ≤ input value |
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |
| 7.Search players by partial player name    |
| 8.Add a new player                         |
| 9.Remove a player with ID                  |
| 10.Update a player with an ID              |
| 11.List all players by a specific position |
| 12.Save data to files                      |
| 13.Load data from files                    |
-----

Input your option: 11
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input player type:
Please enter a valid player type!
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input player type:
Please enter a valid player type!
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input player type:

Please enter a valid player type!
-----
| Choose yes or no                             |
-----
| 0.Yes                                       |
| 1.No (Can choose -1 or 1)                 |
-----

Do you want of continue?: 0
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input player type: asdff
Please enter a valid player type!
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input player type: 1234
Please enter a valid player type!
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input player type: {}{}_
Please enter a valid player type!
Player type: Defender, Winger, Forward, Goalkeeper, Midfielder
Input player type: ))_
Please enter a valid player type!
-----
| Choose yes or no                             |
-----
| 0.Yes                                       |
| 1.No (Can choose -1 or 1)                 |
-----

Do you want of continue?: 1

Please enter to continue

```

Figure 42: When user input wrong position

19. Function 12 – Save data to files

19.1. Happy case

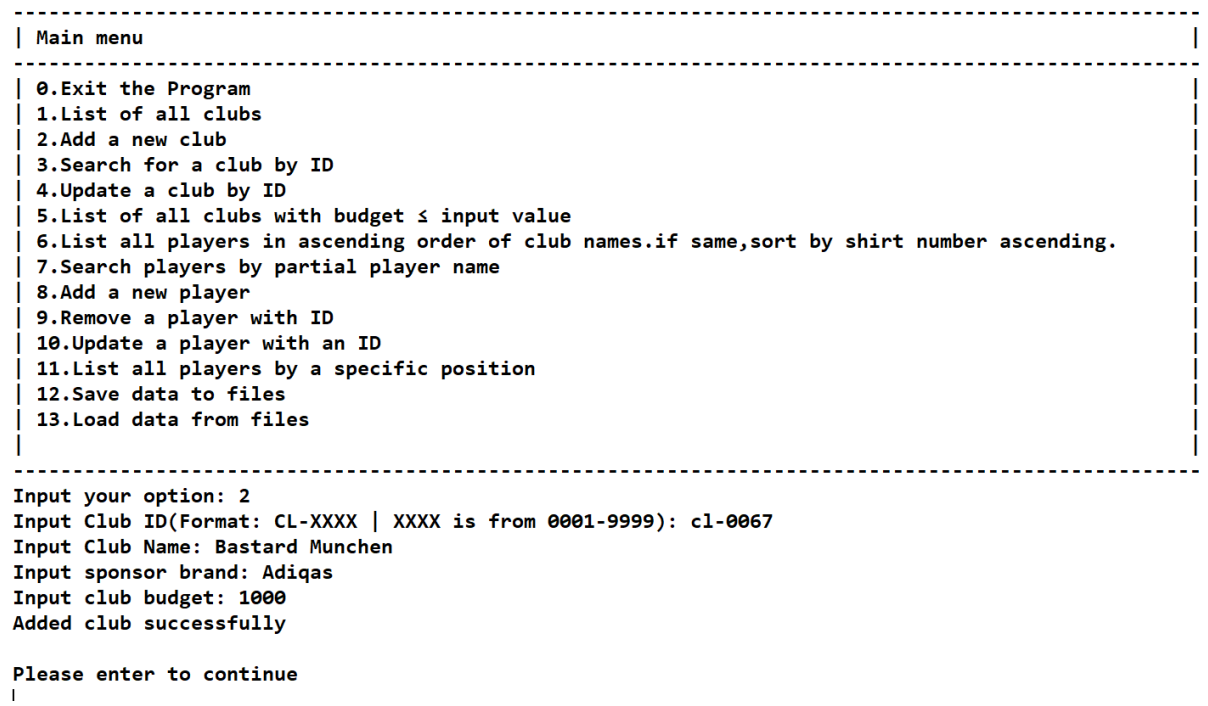
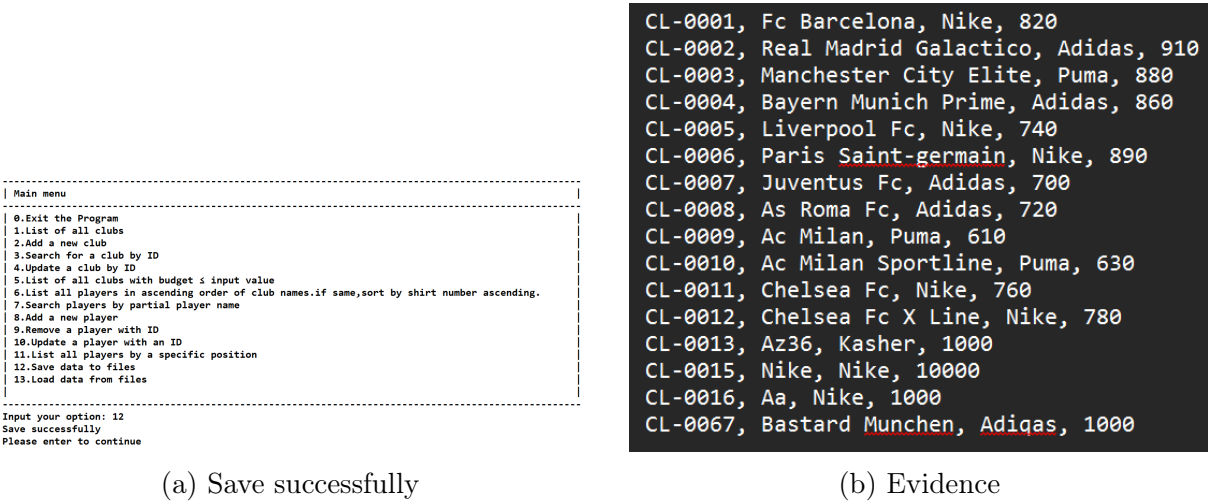


Figure 43: Add player



(a) Save successfully

(b) Evidence

19.2. Unhappy case

Input your option: 4
Input club id: cl-0001

Club ID: CL-0001
Club name: Fc Barcelona
Sponsor Brand: Nike
Budget: 820.0

```
-----
| Update club info menu                                     |
|-----|
| 0.Confirm information and return to menu                 |
| 1.Update club name                                       |
| 2.Update sponsor brand                                   |
| 3.Update budget                                          |
| 4.Refuse changing information and exit                   |
| 5.Confirm information and change club id to update      |
|-----|
-----
```

Input your option: 1
Enter to skip!
Input new club name: Fc Poodle
Press enter to continue!
|

Figure 45: if there is no Player.txt and Club.txt in Data folder

20. Function 13 – Load data from files

20.1. Happy case

```
-----
| Main menu                                               |
|-----|
| 0.Exit the Program                                     |
| 1.List of all clubs                                    |
| 2.Add a new club                                       |
| 3.Search for a club by ID                             |
| 4.Update a club by ID                                 |
| 5.List of all clubs with budget ≤ input value         |
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |
| 7.Search players by partial player name               |
| 8.Add a new player                                     |
| 9.Remove a player with ID                             |
| 10.Update a player with an ID                         |
| 11.List all players by a specific position            |
| 12.Save data to files                                  |
| 13.Load data from files                               |
|-----|
-----
```

Input your option: 13
Load data successfully!

Please enter to continue
|

Figure 46: if there is Player.txt and Club.txt in Data folder

20.2. Unhappy case

```
-----  
| Main menu                                     |  
-----  
| 0.Exit the Program                           |  
| 1.List of all clubs                          |  
| 2.Add a new club                            |  
| 3.Search for a club by ID                   |  
| 4.Update a club by ID                       |  
| 5.List of all clubs with budget ≤ input value |  
| 6.List all players in ascending order of club names.if same,sort by shirt number ascending. |  
| 7.Search players by partial player name      |  
| 8.Add a new player                          |  
| 9.Remove a player with ID                   |  
| 10.Update a player with an ID                |  
| 11.List all players by a specific position   |  
| 12.Save data to files                       |  
| 13.Load data from files                     |  
-----  
Input your option: 13  
Load data failed!  
  
Please enter to continue
```

Figure 47: if there is no Player.txt and Club.txt in Data folder